VECTOR

MICROSAR Ethernet Driver

Technical Reference

Version 6.1.2

Authors	David Walther, Mark Harsch, Benjamin Groebner, Johannes Kolb, Martin Heiker, Deepak Paramashivam
Status	Released

Document Information

History

Author	Date	Version	Remarks
David Walther	2016-12-16	1.00.00	▶ Initial Creation
Mark Harsch	2017-05-10	2.00.00	▶ ESCAN00094983: Extend Core for support of FEC and ENET drivers with extended feature set ▶ Rework to align to most recent technical reference template
Mark Harsch	2017-06-02		▶ Provided information about OS Category 1 interrupt processing
David Walther	2017-07-04	2.01.00	▶ FEATC-1245: FEAT-2151 Extended Ethernet Bus Diagnostic
Benjamin Groebner	2018-02-26	2.01.01	▶ STORY-4103: TASK-65975 Review Integration
Mark Harsch	2018-03-22	3.00.00	▶ STORYC-4202: Introduce support for runtime environment Greenhills INTEGRITY ▶ Minor editorial changes
Mark Harsch	2018-05-25	3.01.00	▶ Correction of review findings
Mark Harsch	2018-06-14	3.02.00	▶ STORYC-5213: Rework configuration of DMA addresses for descriptors and buffers
David Walther	2018-08-17	4.00.00	▶ STORYC-6235: Rework HL/LL-Interface
David Walther	2019-05-08	5.00.00	▶ STORYC-7882: Interface between DrvEth__coreAsr and DrvEthSwitch__coreAsr according to ASR4.4.x (DrvEth__coreAsr) ▶ ESCAN00102223: Description of Service IDs in Technical Reference don't match Service IDs used in the implementation
Johannes Kolb	2019-07-02	5.00.01	▶ ESCAN00103483 Compile error: Eth_30_<MSN>_Ctrl2ProtectedAreaIdMap is undefined ▶ ESCAN00103327 MISRA deviation: MISRA-C:2004 Rule 5.1 – ID 4:0779
Johannes Kolb	2019-07-29	5.01.00	▶ ETH-559 + ETH-561 Extend test suite for CadenceGem
Johannes Kolb	2019-09-09	5.02.00	▶ ETH-739 Migrate CadenceGem to R23
David Walther	2019-09-27	5.03.00	▶ Introduction of Host Controller Interface
Martin Heiker	2019-10-25	5.03.01	▶ ESCAN00104701: Incorrect description of parameter BufIdx in 5.6.2 <User>_TxConfirmationCallout
Johannes Kolb	2019-10-28	6.00.00	▶ ETH-1047 Support Mse102x - QM
David Walther	2020-04-22	6.01.00	▶ ETH-1255 Provide information about dropped frames due to insufficient Tx/Rx

			software buffers
David Walther	2020-07-13	6.01.01	▶ ESCAN00106470: Inconsistent descriptor state possible in transmission path: Memory barrier missing between setting up and triggering transmission
Deepak Paramashivam	2020-08-07	6.01.02	▶ ESCAN00107011: Compiler error: Wrong memory class in Eth_30_Core_Internal_ClearMeasurementData

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_EthernetDriver.pdf	4.1.1
[2]	AUTOSAR	AUTOSAR_SWS_EthernetDriver.pdf	4.3.0
[3]	AUTOSAR	AUTOSAR_SWS_DET.pdf	4.1.1
[4]	AUTOSAR	AUTOSAR_SWS_DEM.pdf	4.1.1
[5]	AUTOSAR	AUTOSAR_BasicSoftwareModules.pdf	V1.0.0
[6]	Vector	TechnicalReference_Eth_<Driver>.pdf	see delivery
[7]	AUTOSAR	AUTOSAR_SWS_NVRAMManager.pdf	4.1.1
[8]	Vector	TechnicalReference_Asr4Rtm.pdf	see delivery
[9]	Vector	TechnicalReference_Microsar_Os_SafeContext_ASR4.pdf	see delivery

Scope of the Document

This technical reference describes the general use of the Ethernet driver basis software. All aspects which are Ethernet controller specific are described in a separate document [6], which is also part of the delivery.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 9

2 Introduction 10
2.1 Architecture Overview 11

3 Functional Description 12
3.1 Features 12
3.1.1 Deviations 12
3.1.2 Additions/ Extensions 13
3.1.3 Platform Dependent Features 13
3.2 Initialization 14
3.2.1 High-Level Initialization 14
3.2.2 Low-Level Initialization 14
3.3 States 14
3.4 Main Functions 15
3.5 Error Handling 15
3.5.1 Development Error Reporting 15
3.5.2 Production Code Error Reporting 17
3.6 MAC Address 17
3.6.1 NV-RAM 17
3.7 Hardware Cancellation 18
3.8 Runtime Measurement 18
3.9 Ethernet Switch Frame Management 19
3.10 Pre-/Post-ControllerInit User-Functions 19
3.11 Rx and Tx Statistics 20
3.12 Retrieval and Reset of Measurement Data 21
3.13 Rx Buffer Warning Level Callout 21
3.14 Driver/Hardware Dependent Features 22
3.14.1 Time Synchronization 23
3.14.1.1 Timer 23
3.14.1.2 Global Time Retrieving 23
3.14.1.3 Ingress Time Stamping 23
3.14.1.4 Egress Time Stamping 24
3.14.2 Quality of Service 26
3.14.3 FQTSS (Traffic Shaping) 27
3.14.4 Checksum Offloading 27
3.14.5 Safe Context 28
3.14.5.1 Privileged Access with OS Callbacks 28

3.14.5.2 Non-Privileged Access by Configuration of a Peripheral Protection Unit 29
3.14.6 Receive Buffer Segmentation 30
3.14.7 Runtime Environment INTEGRITY 30

4 Integration 32
4.1 Embedded Implementation 32
4.2 Compiler Abstraction and Memory Mapping 35
4.3 Exclusive Areas 36
4.4 Memory Mapping 37
4.4.1 Windriver DiabData Compiler 37
4.4.2 Green Hills Compiler 38
4.4.3 Altium TASKING Compiler 38
4.5 Interrupts 39
4.6 Runtime environment INTEGRITY 39
4.6.1 MICROSAR driver 40
4.6.2 IODevice 41

5 API Description 44
5.1 Type Definitions 44
5.2 Structure Definitions 46
5.2.1 Eth_RxStatsType 46
5.2.2 Eth_TxStatsType 48
5.2.3 Eth_TimeStampType 49
5.3 API Table 49
5.3.1 AUTOSAR API 49
5.3.1.1 Eth_30_Core_InitMemory 49
5.3.1.2 Eth_30_Core_Init 50
5.3.1.3 Eth_30_Core_ControllerInit 50
5.3.1.4 Eth_30_Core_SetControllerMode 51
5.3.1.5 Eth_30_Core_GetControllerMode 51
5.3.1.6 Eth_30_Core_GetPhysAddr 52
5.3.1.7 Eth_30_Core_SetPhysAddr 52
5.3.1.8 Eth_30_Core_UpdatePhysAddrFilter 53
5.3.1.9 Eth_30_Core_WriteMii 54
5.3.1.10 Eth_30_Core_ReadMii 54
5.3.1.11 Eth_30_Core_GetCounterState 55
5.3.1.12 Eth_30_Core_ProvideTxBuffer 56
5.3.1.13 Eth_30_Core_Transmit 56
5.3.1.14 Eth_30_Core_Receive 57
5.3.1.15 Eth_30_Core_VTransmit 58

5.3.1.16 Eth_30_Core_TxConfirmation 59
5.3.1.17 Eth_30_Core_GetVersionInfo 59
5.3.1.18 Eth_30_Core_GetRxStats 60
5.3.1.19 Eth_30_Core_GetTxStats 60
5.3.2 AUTOSAR Extension API 61
5.3.2.1 Eth_30_Core_GetAndResetMeasurementData 61
5.3.3 Time Synchronization API 62
5.3.3.1 Eth_30_Core_GetGlobalTime 62
5.3.3.2 Eth_30_Core_EnableEgressTimestamp 63
5.3.3.3 Eth_30_Core_GetEgressTimestamp 63
5.3.3.4 Eth_30_Core_GetIngressTimestamp 64
5.3.4 FQTSS (Traffic Shaping) API 65
5.3.4.1 Eth_30_Core_SetBandwidthLimit 65
5.3.4.2 Eth_30_Core_GetBandwidthLimit 65
5.4 Services used by Ethernet Driver 66
5.5 Callback Functions 67
5.6 Callout Functions 67
5.6.1 <User>_RxIndicationCallout 67
5.6.2 <User>_TxConfirmationCallout 67
5.6.3 Appl_Eth_30_<Driver>_MemoryBarrier 68

6 Configuration 69
6.1 Configuration Variants 69
6.2 Driver specific configuration 69

7 Glossary and Abbreviations 70
7.1 Glossary 70
7.2 Abbreviations 70

8 Contact 71

Illustrations

Figure 2-1 AUTOSAR 4.x Architecture Overview 11
Figure 2-2 Interfaces to adjacent modules of the Ethernet Driver 11
Figure 3-1 Ingress Time-Stamping Sequence Diagram 24
Figure 3-2 Egress Time-Stamping Sequence Diagram 25
Figure 3-3 QoS traffic class configuration 26
Figure 3-4 Peripheral Region Os Identifier 29
Figure 4-1 INTEGRITY AUTOSAR environment project setup 40
Figure 4-2 INTEGRITY kernel project setup 42
Figure 4-3 INTEGRITY integrate file setup 43

Tables

Table 1-1 Component history 9
Table 3-1 Supported AUTOSAR standard conform features 12
Table 3-2 Not supported AUTOSAR standard conform features 13
Table 3-3 Features provided beyond the AUTOSAR standard 13
Table 3-4 Platform dependent Ethernet features 13
Table 3-5 Lower-Level-Initialization 14
Table 3-6 Ethernet Driver states 15
Table 3-7 Ethernet Controller states 15
Table 3-8 Eth_30_<Driver>_MainFunction() description 15
Table 3-9 Service IDs 16
Table 3-10 Errors reported to DET 17
Table 3-11 Errors reported to DEM 17
Table 3-12 Runtime Measurement points 19
Table 3-13 Reception Statistic Counters 21
Table 3-14 Transmission Statistic Counters 21
Table 3-15 Vector Driver Feature Set Support 22
Table 3-16 Traffic class to ring priority mapping 26
Table 3-17 Checksum Offloading Capability 27
Table 3-18 Peripheral Region address range 29
Table 4-1 List of delivered files 35
Table 4-2 Compiler abstraction and memory mapping 36
Table 4-3 Exclusive areas 36
Table 5-1 Type definitions 46
Table 5-2 Eth_RxStatsType Structure 48
Table 5-3 Eth_TxStatsType Structure 49
Table 5-4 Eth_TimeStampType Structure 49
Table 5-5 Eth_30_Core_InitMemory 49
Table 5-6 Eth_30_Core_Init 50
Table 5-7 Eth_30_Core_ControllerInit 51
Table 5-8 Eth_30_Core_SetControllerMode 51
Table 5-9 Eth_30_Core_GetControllerMode 52
Table 5-10 Eth_30_Core_GetPhysAddr 52
Table 5-11 Eth_30_Core_SetPhysAddr 53
Table 5-12 Eth_30_Core_UpdatePhysAddrFilter 54
Table 5-13 Eth_30_Core_WriteMii 54
Table 5-14 Eth_30_Core_ReadMii 55
Table 5-15 Eth_30_Core_GetCounterState 56
Table 5-16 Eth_30_Core_ProvideTxBuffer 56
Table 5-17 Eth_30_Core_Transmit 57

Table 5-18 Eth_30_Core_Receive ... 58
Table 5-19 Eth_30_Core_VTransmit ... 59
Table 5-20 Eth_30_Core_TxConfirmation ... 59
Table 5-21 Eth_30_Core_GetVersionInfo ... 60
Table 5-22 Eth_30_Core_GetRxStats ... 60
Table 5-23 Eth_30_Core_GetTxStats ... 61
Table 5-24 Eth_30_Core_GetAndResetMeasurementData ... 62
Table 5-25 Eth_30_Core_GetGlobalTime ... 62
Table 5-26 Eth_30_Core_EnableEgressTimestamp ... 63
Table 5-27 Eth_30_Core_GetEgressTimestamp ... 64
Table 5-28 Eth_30_Core_GetIngressTimestamp ... 65
Table 5-29 Eth_30_Core_SetBandwidthLimit ... 65
Table 5-30 Eth_30_Core_GetBandwidthLimit ... 66
Table 5-31 Services used by the Ethernet Driver ... 66
Table 5-32 <User>_RxIndicationCallout ... 67
Table 5-33 <User>_TxConfirmationCallout ... 68
Table 5-34 Appl_Eth_30_<Driver>_MemoryBarrier ... 68
Table 7-1 Glossary ... 70
Table 7-2 Abbreviations ... 70

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
01.00.xx	▶ Initial Component Release
02.00.xx	▶ Extensions to support VTT specific driver
03.00.xx	▶ Extensions to support FEC and ENET specific drivers with their extended feature set
03.01.xx	▶ FEAT-2151: Extended Ethernet Bus Diagnostic
03.02.xx	▶ Changes due to DrvEth_S6J3xEthAsr migration
03.03.xx	▶ Introduce support for GNU compiler
04.00.xx	▶ Introduced support for runtime environment INTEGRITY
04.01.xx	▶ Read/write/modify access of hardware interfacing data structures completely handled in INTEGRITY kernel-space
04.02.xx	▶ DMA addresses for buffers and descriptors are completely handled in kernel-space when using INTEGRITY ▶ Descriptor and buffer part of driver related content of MemMap.h is now generated into Eth_30_<Driver>_MemMap.h
05.00.xx	▶ Process 3.0 refactoring and maintenance
06.00.xx	▶ SafeBSW for Eth
06.01.xx	▶ SafeBSW with changes due to DrvEth_CadenceGemAsr migration (QM)
06.02.xx	▶ SafeBSW with changes due to DrvEth_CadenceGemAsr migration (ASIL)
07.00.xx	▶ Interface between DrvEth and DrvEthSwitch according to ASR4.4.x
07.01.xx	▶ ETH-739 Migrate CadenceGem to R23
08.00.xx	▶ Introduction of Host-Controller Interface and support of Mse102x
08.01.xx	▶ Support for Tms570 ▶ ESCAN00104701: Incorrect description of parameter BufIdx in 5.6.2 <User>_TxConfirmationCallout
08.02.xx	▶ ETH-1047 Support for Mse102x - QM
09.00.xx	▶ ETH-1099 DrvEth__coreAsr – Merge 7.xx.xx into trunk.
09.01.xx	▶ ETH-1255 Provide information about dropped frames due to insufficient Tx/Rx software buffers

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module Ethernet Driver as specified in [1] and the extensions of the Vector Ethernet drivers.

This document describes the functionalities provided by the whole range of Vector Ethernet drivers. Not any driver supports all functionalities. Please refer to [6] to check, which functionality is available and can be used with the specific driver.

Supported AUTOSAR Release*:	4.1.1	
Supported Configuration Variants:	pre-compile	
Vendor ID:	ETH_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	ETH_MODULE_ID	88 decimal (according to ref. [5])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The Ethernet Driver provides hardware independent access to configure and control Ethernet Controllers integrated in the microcontroller (MCU) or connected to it via an external interface like for example SPI.

2.1 Architecture Overview

Figure 2-1 shows where the Ethernet Driver is located in the AUTOSAR architecture.

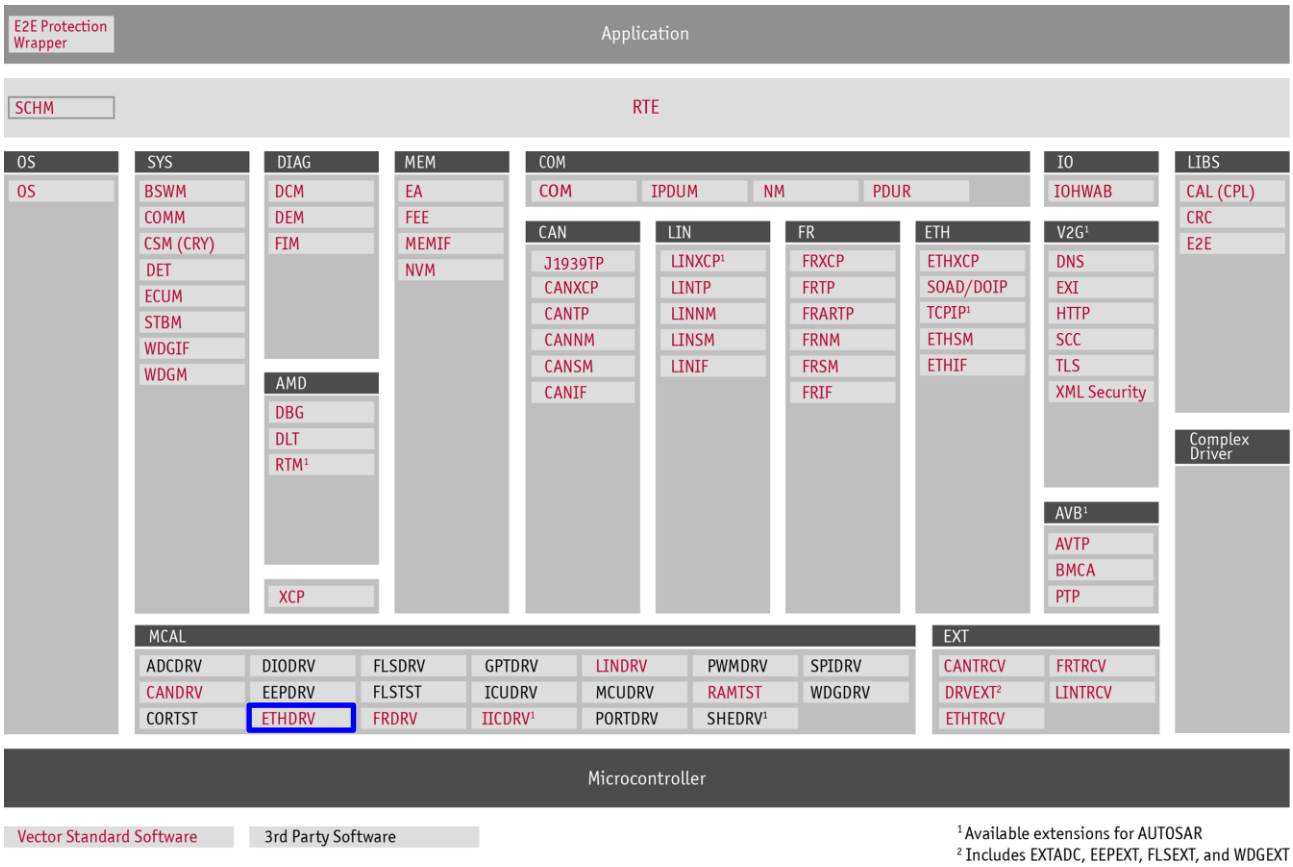


Figure 2-1 AUTOSAR 4.x Architecture Overview

Figure 2-2 shows the interfaces to adjacent modules of the Ethernet Driver. These interfaces are described in chapter 0.

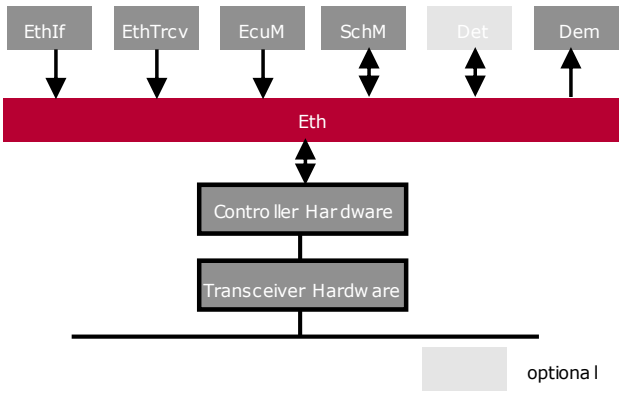


Figure 2-2 Interfaces to adjacent modules of the Ethernet Driver

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the whole range of Vector Ethernet Drivers.

For extensions beyond or limitation of the described functionalities please see [6] of the specific driver.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further Ethernet Driver functionality beyond the AUTOSAR standard. The corresponding features are listed in the tables

> Table 3-3 Features provided beyond the AUTOSAR standard

> Table 3-4 Platform dependent Ethernet features

The following features specified in [1] are supported:

Supported AUTOSAR Standard Conform Features
Initialization of the Ethernet Controller
Activation/Deactivation of the Ethernet Controller during runtime
Reception/Transmission of Ethernet Frames according to IEEE802.3
Retrieve current active MAC address
Set MAC address
Modification of the MAC address filter
Access of hardware connected via MDIO interface according to IEEE802.3 clause 22
Retrieval of statistic counters located in the register space of the Ethernet Controller
Retrieval of the modules version info
Report development errors to DET
Report production errors to DEM
Operation in interrupt mode
Operation in polling mode
Multiple Ethernet controller operation
Multiple Ethernet driver operation

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Deviations

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features
Link-time configuration variant

Not Supported AUTOSAR Standard Conform Features
Post-build configuration variant

Table 3-2 Not supported AUTOSAR standard conform features

3.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond the AUTOSAR Standard
Storage of MAC address in NV-RAM for re-initialization after MCU power-cycle
Extended Buffer Configuration
Runtime Measurement
Flexible Interrupt configuration
Pre-/Post-ControllerInit User-Callouts
Notification callout in case all reception buffers are occupied by the software
Time Synchronization
Quality of Service
FQTSS (Traffic Shaping)
Checksum Offloading
Receive Buffer Segmentation
Rx and Tx Statistics
Retrieval and Reset of Measurement Data
Runtime environment INTEGRITY

Table 3-3 Features provided beyond the AUTOSAR standard

3.1.3 Platform Dependent Features

Table 3-4 lists all driver/hardware dependent features provided by Vector drivers.

Please refer to [6] for details if the noted features are available by the specific driver used.

Features Provided Driver/Hardware Specific
Time Synchronization
Quality of Service
FQTSS (Traffic Shaping)
Checksum Offloading
Receive Buffer Segmentation
Runtime environment INTEGRITY

Table 3-4 Platform dependent Ethernet features

3.2 Initialization

3.2.1 High-Level Initialization

The Ethernet Driver is initialized by calling the `Eth_30_<Driver>_Init()` service with `NULL_PTR` as configuration pointer due to Pre-Compile-Configuration-Variant only support

The Ethernet controller itself is initialized by calling the `Eth_30_<Driver>_ControllerInit()` service for the respective controller to be initialized.



Note
When using a MICROSAR SIP the initialization is done by the stack itself without any user interaction.



Caution
If startup code doesn't initialize the RAM and therefore some data isn't set to a predefined value the Ethernet Driver relies on (e.g. zero initialized data), the function `Eth_30_<Driver>_InitMemory()` must be called during startup. This function sets the predefined values usually set by the startup code.

3.2.2 Low-Level Initialization

The Ethernet Driver relies on the proper configuration of the MCU, which must be done by the MCAL modules. Table 3-5 contains information about the modules involved and what preconditions must be provided by them for proper functionality of the Ethernet Controller.

AUTOSAR MCAL module	Preconditions
Port	Initialization of the MCU pins for the MII interface to the Ethernet Transceiver/Ethernet Switch Port.
Mcu	Initialization of the clocks provided to the Ethernet Controller.

Table 3-5 Lower-Level-Initialization

3.3 States

The Ethernet Driver adopts the states described in the following table:

State	Description
`ETH_STATE_UNINIT`	The driver wasn't initialized yet and all its state variables are set to an undefined value.
`ETH_STATE_INIT`	The driver was initialized and its module global state variables are set to a defined value allowing starting initialization of the Ethernet controller.
`ETH_STATE_ACTIVE`	At least one Ethernet controller and its related states were initialized and ready for operation.

Table 3-6 Ethernet Driver states

The Ethernet Controller adopts the states described in the following table:

State	Description
ETH_MODE_DOWN	The Ethernet controller is turned off and can't be used for communication.
ETH_MODE_ACTIVE	The Ethernet controller is turned on and able to transmit/receive frames.

Table 3-7 Ethernet Controller states

Most APIs of the driver are only allowed to be called in specific states either of the driver or the Ethernet Controller. For details see the service descriptions in 0.

3.4 Main Functions

The Ethernet Driver has one main function described in the following table:

Eth_30_<Driver>_MainFunction()		
Existence	Timing	Description
Always	Scheduled according to the period provided by the configuration parameter EthMainFunctionPeriod.	Processes driver specific task described in [6].

Table 3-8 Eth_30_<Driver>_MainFunction() description



Note
The main function isn't declared by the Ethernet Driver itself but declaration is provided by the SchM during generation with DaVinci Configurator PRO.

3.5 Error Handling

3.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service Det_ReportError() as specified in [3], if development error reporting is enabled (i.e. pre-compile parameter ETH_30_<DRIVER>_DEV_ERROR_DETECT==STD_ON).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service Det_ReportError().

The reported Ethernet Driver ID is 88.

The reported service IDs identify the services which are described in [3]. Table 3-9 presents the service IDs and the related services:

Service ID	Service
0x01	ETH_30_<DRIVER>_SID_INIT
0x02	ETH_30_<DRIVER>_SID_CONTROLLER_INIT
0x03	ETH_30_<DRIVER>_SID_SET_CONTROLLER_MODE
0x04	ETH_30_<DRIVER>_SID_GET_CONTROLLER_MODE
0x05	ETH_30_<DRIVER>_SID_WRITE_MII
0x06	ETH_30_<DRIVER>_SID_READ_MII
0x07	ETH_30_<DRIVER>_SID_GET_COUNTER_STATE
0x08	ETH_30_<DRIVER>_SID_GET_PHYS_ADDR
0x09	ETH_30_<DRIVER>_SID_PROVIDE_TX_BUFFER
0x0A	ETH_30_<DRIVER>_SID_TRANSMIT
0x0B	ETH_30_<DRIVER>_SID_RECEIVE
0x0C	ETH_30_<DRIVER>_SID_TX_CONFIRMATION
0x0D	ETH_30_<DRIVER>_SID_GET_VERSION_INFO
0x0E	ETH_30_<DRIVER>_SID_GET_RX_STATS
0x0F	ETH_30_<DRIVER>_SID_GET_TX_STATS
0x12	ETH_30_<DRIVER>_SID_UPDATE_PHYS_ADDR_FILTER
0x13	ETH_30_<DRIVER>_SID_SET_PHYS_ADDR
0x30	ETH_30_<DRIVER>_TIME_SYNC_SID_GET_GLOBAL_TIME
0x31	ETH_30_<DRIVER>_TIME_SYNC_SID_ENABLE_EGRESS_TIMESTAMP
0x32	ETH_30_<DRIVER>_TIME_SYNC_SID_GET_EGRESS_TIMESTAMP
0x33	ETH_30_<DRIVER>_TIME_SYNC_SID_GET_INGRESS_TIMESTAMP
0x40	ETH_30_<DRIVER>_SID_SET_BANDWIDTH_LIMIT
0x41	ETH_30_<DRIVER>_SID_GET_BANDWIDTH_LIMIT

Table 3-9 Service IDs

The errors reported to DET are described in Table 3-10:

Error Code		Description
0x00	ETH_30_<DRIVER>_E_NO_ERROR	No error occurred
0x01	ETH_30_<DRIVER>_E_INV_CTRL_IDX	API called with wrong controller index
0x02	ETH_30_<DRIVER>_E_NOT_INITIALIZED	API called while module was not initialized correctly
0x03	ETH_30_<DRIVER>_E_INV_POINTER	API called with wrong pointer parameter (NULL_PTR)
0x04	ETH_30_<DRIVER>_E_INV_PARAM	API called with invalid parameter
0x05	ETH_30_<DRIVER>_E_INV_CONFIG	Initialization triggered for an unknown configuration

Error Code		Description
0x06	ETH_30_<DRIVER>_E_INV_MODE	API called while module was in an invalid mode
0x07	ETH_30_<DRIVER>_E_INV_ALIGNMENT	Invalid alignment of buffer or descriptor

Table 3-10 Errors reported to DET

3.5.2 Production Code Error Reporting

By default, production code related errors are reported to the DEM using the service `Dem_ReportErrorStatus()` as specified in [4].

The errors reported to DEM are described in Table 3-11:

Error Code	Description
ETH_E_ACCESS	Accessing the controller failed

Table 3-11 Errors reported to DEM

3.6 MAC Address

The MAC address of a controller may be configured to a default value via the MAC address configuration parameter within the configuration tool. Additionally, the user may call the API `Eth_30_<Driver>_SetPhysAddr` to change the MAC address at runtime.



Caution
In case the user does not provide a default address within the configuration tool, the address is undefined after ECU startup. Therefore, it is essential that the user sets the MAC at runtime before any Ethernet communication starts!

The default behavior of `Eth_30_<Driver>_SetPhysAddr` is non-persistent. The user must provide the MAC address each time after an ECU restart (in case no default is configured within the configuration tool). An extended, persistent behavior can be enabled via the feature “Enable MAC address write access” discussed in the next section.

3.6.1 NV-RAM

The feature “Enable MAC address write access” enables NvM support for the API `Eth_30_<Driver>_SetPhysAddr`. In this case the MAC address gets written into a NV-RAM block and is loaded at controller initialization. This ensures that the MAC address is persistent even after a system restart.

The NV-RAM block for a MAC must have a length of 6 bytes. The initial value is configured via the MAC address configuration parameter within the configuration tool. The NV-RAM block must be managed by the AUTOSAR NV-Manager (NvM) and is addressed by a NvM block descriptor (refer to [7]).

The NV-RAM blocks must be processed during the `NvM_ReadAll()` and `NvM_WriteAll()` function calls.

The symbols for the ROM and RAM mirrors are listed below

- ROM default block: `Eth_30_<Driver>_VPhysSrcAddrRomDefault_<CtrlIdx>`
- RAM mirror: `Eth_30_<Driver>_VPhysSrcAddr_<CtrlIdx>`

where `<CtrlIdx>` is the index of the controller.

3.7 Hardware Cancellation

The hardware cancellation feature can be applied for different hardware operations by setting the corresponding cycle number. While performing these operations a counter is incremented in a loop in every cycle until either the operation is finished or the configured maximum value of the cycle number has been reached. In this case the operation has failed. The configurable maximum cycle counter values are:

- Controller Reset Loop Cycles: Specifies the number of loop cycles after which the controller reset timeout occurs.
- Controller MII Loop Cycles: Specifies the number of loop cycles after which the timeout for MII accesses occurs.
- Controller Loop Cycles: Specifies the number of loop cycles after which other hardware dependent loop timeouts occur.



Caution
The Loop Cycles Counter is not related to any time base! Therefore the time elapsing is highly platform and configuration dependent (e.g. CPU clock dependent)!

3.8 Runtime Measurement

Runtime measurement allows to measure the processing time of specific functions of the driver. Therefore, so called measurement points are used. These points are automatically created in the Rtm module, if the module is contained in the configuration.

This description only shows the Ethernet drivers specific aspect of the runtime measurement. For a more detailed description of the runtime measurement feature and the Rtm module providing the service see [8].

Dependent on the Ethernet controller core integrated on the derivative used and the features enabled, measurement points are provided.

Measurement Point	Description
`Eth_30_<Driver>_ControllerInit`	Measures the runtime of the API `Eth_30_<Driver>_ControllerInit()`. The

Measurement Point	Description
	API is used for controller initialization during runtime.

Table 3-12 Runtime Measurement points



Note
The runtime measurement is enabled by setting `Runtime Measurement Support` in the general settings of the Ethernet driver.

3.9 Ethernet Switch Frame Management

Some Ethernet Switches provide a functionality called frame management. This feature makes it possible to transfer Meta data related to an Ethernet frame over the MII interface. This Meta data is either embedded into the Ethernet Frame itself or transferred as a consecutive Ethernet frame. To allow the Ethernet Driver to process the Ethernet frames without the knowledge which kind of mechanism is used, an interaction between Ethernet Driver and Ethernet Switch Driver is needed.

For a more detailed description of the interface please refer to the Ethernet Switch Drivers technical reference.



Note
The feature is implicitly activated if an Ethernet Switch Driver is contained in the configuration and uses the Ethernet Controller as frame management interface.

3.10 Pre-/Post-ControllerInit User-Functions

The driver allows to inject integration code in the processing of the `Eth_30_<Driver>_ControllerInit()` API.

This integration code is either executed before the actual code of the `Eth_30_<Driver>_ControllerInit()` (Pre-ControllerInit User-Function), after the actual code (Post-ControllerInit User-Function) or both.

The functions can be used to integrate derivative specific code that must be executed for a proper operation of the Ethernet Controller. Such code could be setting registers in port or clock modules, which are needed to configure the xMII interface between the Ethernet Controller and the Ethernet PHY properly.

The User-Functions will be generated into the `Eth_30_<Driver>_Lcfg.c` file and contain a so-called User-Block-Comment where integration code can be provided, which is persisted and not overridden if the `Eth_30_<Driver>_Lcfg.c` file is generated again.

The function names are `Eth_30_<Driver>_PreControllerInitCallout()` and `Eth_30_<Driver>_PostControllerInitCallout()`.

3.11 Rx and Tx Statistics

With the functions `Eth_30_<Driver>_GetRxStats` and `Eth_30_<Driver>_GetTxStats` the lists of counter values that can be seen in Table 3-13 and Table 3-14 can be obtained from the hardware. Please note, that if a realistic counter value cannot be obtained from the underlying hardware platform, the API returns a special value. See sections 5.2.1 and 5.2.2 for details on the used data type.

Counter Value	Description
Rx Drop Events	The total number of events in which packets were dropped due to lack of ressources.
Rx Octets	The total number of octets of data (including those in bad packets) received on the network (excluding framing bits but including FCS octets).
Rx Packets	The total number of packets (including bad packets, broadcast packets and multicast packets) received.
Rx Broadcast Packets	The total number of good packets received that were directed to the broadcast address.
Rx Multicast Packets	The total number of good packets received that were directed to a multicast address.
Rx Crc/Alignment Errors	The total number of packets received that had a length between 64 and 1518 octets that had either a bad Frame Check Sequence (FCS) with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error).
Rx Undersize Packets	The total number of packets received that were less than 64 octets long (excluding framing bits, but including FCS octets) and were otherwise well formed.
Rx Oversize Packets	The total number of packets received that were longer than 1518 octets long (excluding framing bits, but including FCS octets) and were otherwise well formed.
Rx Fragments	The total number of packets received that were less than 64 octets in length (excluding framing bits but including FCS octets) and had either a bad FCS with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error).
Rx Jabbers	The total number of packets received that were longer than 1518 octets in length and had either a bad FCS with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error).
Rx Collisions	The best estimate of the total number of collisions on this Ethernet segment.
Rx Packets 64 Octets	The total number of packets (including bad packets) received that were 64 octets in length.
Rx Packets 65 to 127 Octets	The total number of packets (including bad packets) received that were between 65 and 127 octets in length.
Rx Packets 128 to 255 Octets	The total number of packets (including bad packets) received that were between 128 and 255 octets in length.
Rx Packets 256 to 511 Octets	The total number of packets (including bad packets) received that were between 256 and 511 octets in length.

Counter Value	Description
Rx Packets 512 to 1023 Octets	The total number of packets (including bad packets) received that were between 512 and 1023 octets in length.
Rx Packets 1024 to 1518 Octets	The total number of packets (including bad packets) received that were between 1024 and 1518 octets in length.
Rx Unicast Frames	The number of subnetwork-unicast packets delivered to a higher level protocol.

Table 3-13 Reception Statistic Counters

Counter Value	Description
Tx Number Of Octets	The total number of octets transmitted out of the interface, including framing characters.
Tx Non-unicast Packets	The total number of packets that higher level protocols requested to be transmitted to a non-unicast (i.e., a subnetwork-broadcast or subnetwort-multicast) address, including those that were discarded or not sent.
Tx Unicast Packets	The total number of packets that higher level protocols requested to be transmitted to a subnetwork-unicast address, including those that were discarded or not sent.

Table 3-14 Transmission Statistic Counters

3.12 Retrieval and Reset of Measurement Data

The driver automatically detects and counts certain important events in the reception and transmission process:

- The amount of times, a Tx buffer request was denied due to a lack of Tx buffers available
- The amount of times, a Rx frame is dropped due to a lack of free Rx buffers available. This counter is only supported, if a corresponding counter register is available on the used hardware. If the counter register is not present, the function `Eth_30_<Driver>_GetAndResetMeasurementData` will return E_NOT_OK for this counter.
- The amount of times, all Rx buffers are occupied during the reception of a frame. This means, that the currently processed frame occupied the last available buffer. Additionally, an optional callout is invoked when this condition is detected. See section 3.13 for more details.

In order to retrieve the counter values or reset them, the user can invoke the API `Eth_30_<Driver>_GetAndResetMeasurementData`. The supported measured data types and the corresponding index can be seen in section 5.1 in Table 5-1 in the definition of the type `Eth_MeasurementIdxType`.

For details on `Eth_30_<Driver>_GetAndResetMeasurementData`, see the API description in Table 5-24.

3.13 Rx Buffer Warning Level Callout

The Rx Buffer Warning Level Callout is an optional feature. If it is enabled, whenever a frame reception is detected on driver level, the available reception buffers are checked for

their status. If the condition is detected, that none of the available configured reception buffers is currently free for the hardware to copy incoming frame data to, a user-defined callout is triggered. If the condition occurs, it could mean that Rx frames are lost.



Caution
This feature cannot detect frame losses by itself, since this is dependent on the amount of incoming traffic during the time the buffers are blocked and additional buffer capabilities within the used hardware (e.g. Rx-FIFO size).

Actual Rx frame losses can only be detected by the hardware itself, and could be read out if a corresponding statistic register is available by using `Eth_<Driver>_GetCounterState`, `Eth_<Driver>_GetRxStats` or `Eth_30_<Driver>_GetAndResetMeasurementData`.

The Rx Buffer Warning Level Callout can be used during development, to detect if the blocked condition ever occurs, which can indicate that there is a need for the configuration of more reception buffers to process the expected incoming traffic. Additionally, it could be used to create an application specific, user defined statistic about how frequent this condition occurs during runtime in the target system, e.g. to perform a diagnostic readout.

The feature can be enabled by configuring a proper function name for the callout in Eth/EthGeneral/EthRxBufWarnLvlCallout.

3.14 Driver/Hardware Dependent Features

The following sections introduce features which are contained in the software provided by Vector but are not supported on each driver or hardware derivative.

Please refer to [6] section 3.1. where following table can be found that summarizing the availability of the features by the respective driver or on the specific hardware derivative used.

Feature	Driver/Hardware support - = not available ■ = available on specific derivatives ■ = available
Time Synchronization	■
Quality of Service	■
FQTSS (Traffic Shaping)	■
Checksum Offloading	■
Safe Context	■
Receive Buffer Segmentation	-
Runtime environment INTEGRITY	■

Table 3-15 Vector Driver Feature Set Support

3.14.1 Time Synchronization

The time synchronization allows synchronizing the Ethernet Controllers timer to a respective time base by retrieving timestamps of Ethernet frames on transmission and reception.



Note
The feature is enabled by setting ‘Enable PTP’ in the Ethernet Controller configuration in DaVinci Configurator.

3.14.1.1 Timer

For the support of PTP the Ethernet Controller core implements a timer which maintains the local time. This time base is used to timestamp the frames.



Note
For proper operation of the timer an external clock must be provided. Please refer to the peripheral clock configuration chapter of the Technical Reference provided for the microcontroller derivative used.

The clock frequency provided additionally must be defined in the controller configuration in DaVinci Configurator by setting the parameter ‘PTP Reference Clock Frequency’.

3.14.1.2 Global Time Retrieving

An upper layer is able to retrieve the global time. Therefore the API `Eth_30_<Driver>_GetGlobalTime()` is provided.



Note
This API can be called without the restriction to the Rx Indication or Tx Confirmation context.

3.14.1.3 Ingress Time Stamping

If an incoming Ethernet frame is time stamped by the Ethernet Controller, an upper layer is able to retrieve this timestamp by calling the API `Eth_30_<Driver>_GetIngressTimestamp()`.

Caution
The API `Eth_30_<Driver>_GetIngressTimestamp()` must be called in the Rx Indication context to retrieve a proper timestamp for the received Ethernet frame. Otherwise the API will return with the negative return value `E_NOT_OK` and the timestamp is invalid.

Figure 3-1 illustrates the sequence with an EthTSyn (GPtp) upper layer:

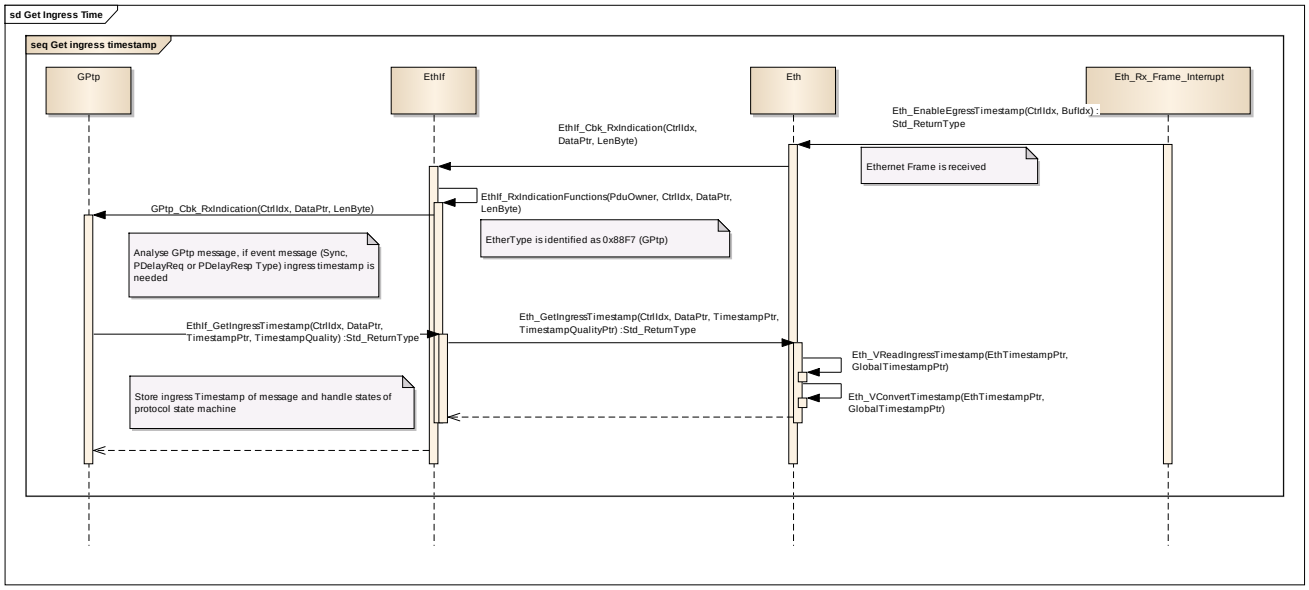


Figure 3-1 Ingress Time-Stamping Sequence Diagram

3.14.1.4 Egress Time Stamping

The time stamping of outgoing Ethernet frames must be triggered by the upper layer itself.

To achieve the time stamping the upper layer must obtain a transmit buffer by calling `Eth_30_<Driver>_ProvideTxBuffer()`. Afterwards the buffer provided is locked and time stamping for this buffer can be enabled by calling `Eth_30_<Driver>_EnableEgressTimestamp()`. The transmission now can be trigger with the API `Eth_30_<Driver>_Transmit()`.

The time stamp itself is retrieved on transmission indication by calling `Eth_30_<Driver>_GetEgressTimestamp()`.

Caution
The API `Eth_30_<Driver>_GetEgressTimestamp()` must be called in the TX Confirmation context to retrieve a proper timestamp for the transmitted Ethernet Frame. Otherwise the API will return with the negative return value `E_NOT_OK` and the timestamp is invalid.

Caution
On transmit of an Ethernet frame to be time-stamped a TX Confirmation must be requested by the upper layer.

Figure 3-2 illustrates the sequence with an EthTSyn (GPtp) upper layer:

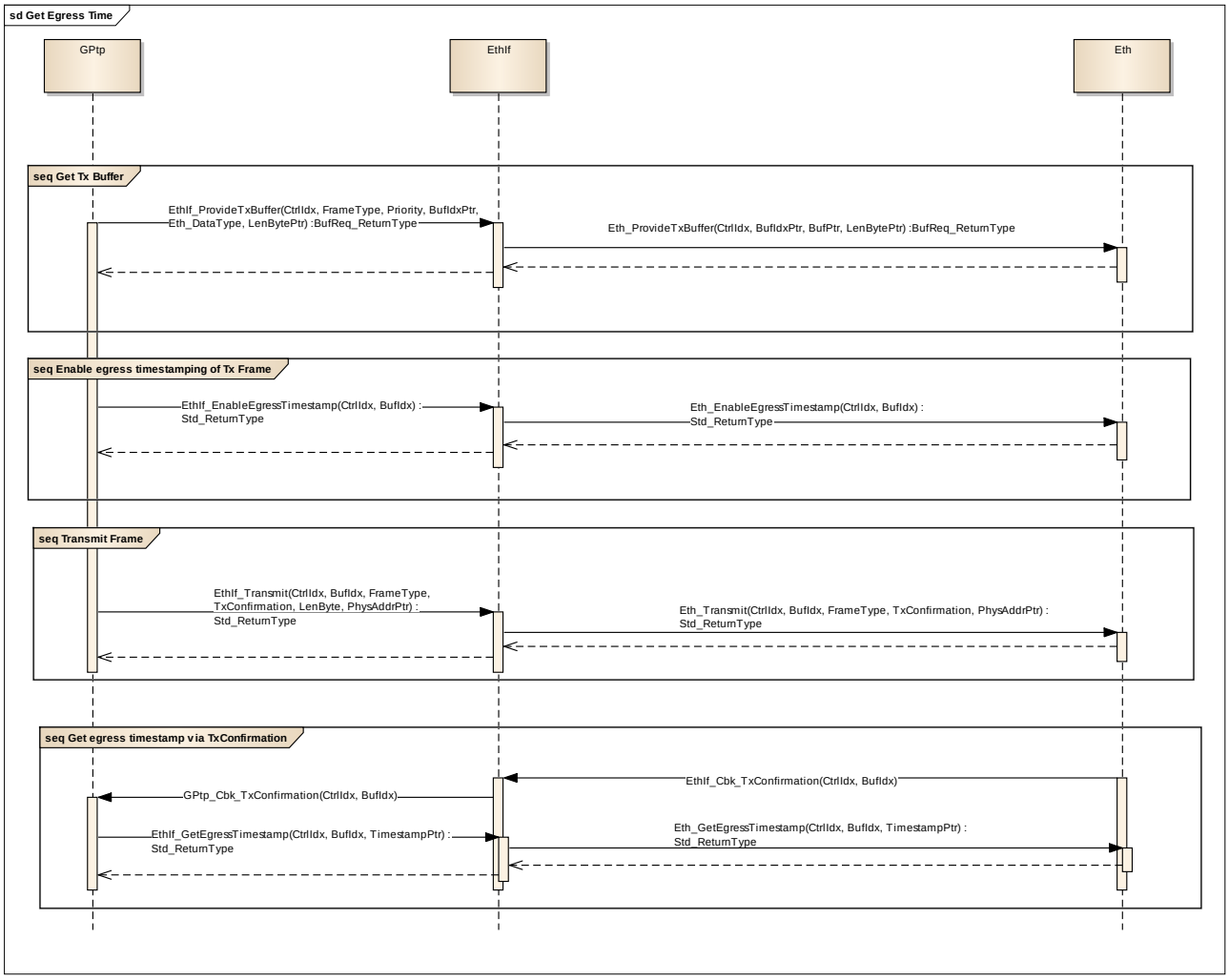


Figure 3-2 Egress Time-Stamping Sequence Diagram

3.14.2 Quality of Service

Quality of Service (QoS) allows prioritizing the Ethernet traffic dependent on the priority of a VLAN tagged frame.

Note
Quality of Service can only be applied on VLAN tagged traffic because it relies on the priority encoded in the VLAN TCI (Tag Control Information).

The following example provides three traffic classes: The best effort class-, class-1- and class-2-traffic, where best effort has the lowest priority and class 2 has the highest priority.

The different classes can be identified in the configuration tool by the parameter `Priority` (see Figure 3-3) and Table 3-16.

Traffic Class	Priority
Best effort	0
Class 1	1
Class 2	2

Table 3-16 Traffic class to ring priority mapping

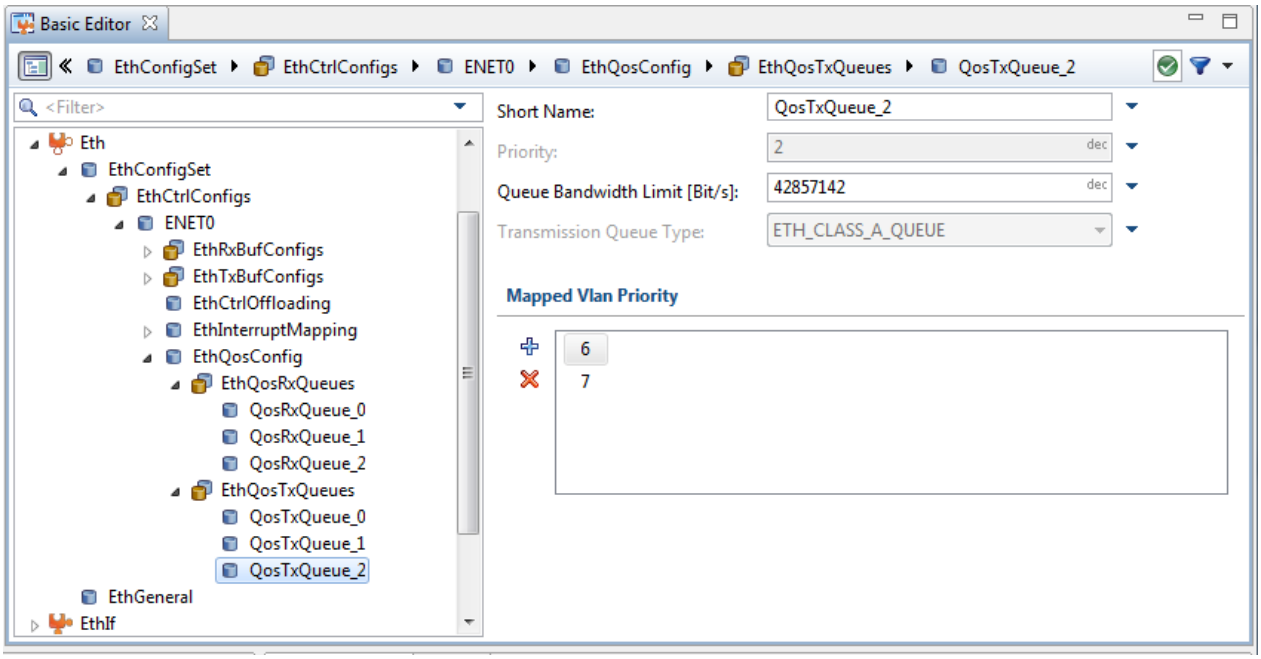


Figure 3-3 QoS traffic class configuration

Like mentioned before, the traffic classification is done with the help of the priority encoded by the VLAN tag. To achieve this, the VLAN priority is used during configuration time to assign traffic to a specific traffic class. This can be done for the Transmission- and for the

Reception-Path as well. In the screenshot, illustrated by Figure 3-3, for example, traffic class 2 on Transmission-Path has assigned all frames sent with the VLAN priority 6 and 7.

3.14.3 FQTSS (Traffic Shaping)

The FQTSS (Forwarding and Queuing Enhancements for Time-Sensitive Streams) support allows applying traffic shaping for Ethernet traffic on the Transmission-Path. The mechanism is based on the QoS traffic classes described in the section 3.14.2.

The traffic classes are assigned to a traffic shaper, which defines an idle slope to shape the Ethernet traffic. The driver allows configuring the shapers in a static way during configuration time by providing the desired bandwidth in [bit/s].

Additionally the current bandwidth limit can be manipulated or retrieved during runtime by using the APIs `Eth_30_<Driver>_SetBandwidthLimit()` and `Eth_30_<Driver>_GetBandwidthLimit()`.



Note
FQTSS (Traffic Shaping) can be activated by configuration.

3.14.4 Checksum Offloading

Checksum Offloading allows calculating and checking checksums of protocols of the TcpIp stack. In principle following checksum calculation can be offloaded to hardware. However not each hardware is capable to calculate all checksums. For details regarding the used driver/hardware please see [6].

Protocol	Description
ICMPv4	Calculation of ICMPv4 checksum
IPv4	Calculation of Ipv4 checksum
UDP over IPv4	Calculation of UDP checksum when embedded in IPv4 protocol
TCP over IPv4	Calculation of TCP checksum when embedded in IPv4 protocol
ICMPv6	Calculation of ICMPv6 checksum
IPv6 with Routing Header Option	Calculation of checksum when IPv6 Routing Header Option is involved
IPv6 with Destination Header Option	Calculation of checksum when IPv6 Destination Header Option is involved
IPv6 with Hop by Hop Header Option	Calculation of checksum when IPv6 Hop by Hop Header Option is involved
UDP over IPv6	Calculation of UDP checksum when embedded in IPv6 protocol
TCP over IPv6	Calculation of TCP checksum when embedded in IPv6 protocol

Table 3-17 Checksum Offloading Capability



Note

Checksum Offloading can be activated by configuration.

3.14.5 Safe Context

CPUs on certain platforms can support different operation modes (e.g. User and Privileged/Supervisor mode). Depending on the used derivative the R/W access to Ethernet hardware registers can be restricted to Privileged mode only. Additionally, depending on the system design, the Ethernet Driver may not be allowed to switch into a Privileged mode by itself. Therefore, the protected registers must be accessed from outside of the Ethernet Driver by another software layer (e.g. the Operating System) which is able to switch into the required Privileged mode.

3.14.5.1 Privileged Access with OS Callbacks

The software layer (e.g. the Operating System) which is responsible to handle the privileged access to protected registers must add an area define according to the configured ‘Peripheral Region Os Identifier’ parameter value in the Ethernet controller configuration in DaVinci Configurator Pro.

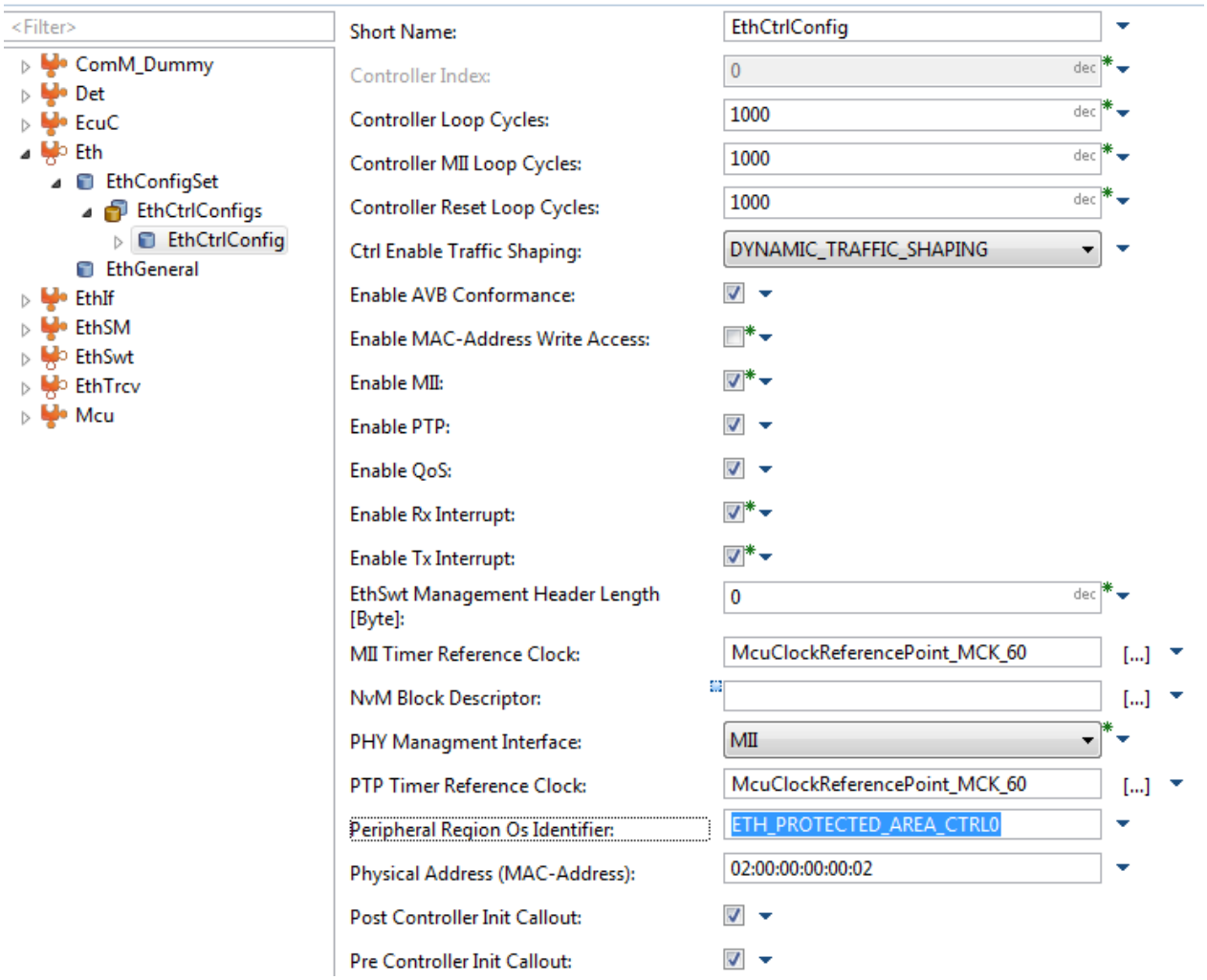


Figure 3-4 Peripheral Region Os Identifier

For each used ‘Peripheral Region Os Identifier’ the following register address space has to be located inside the privileged region that can be configured within the Operating System module in DaVinci Configurator Pro:

Physical address
ETH base address + 0x0000 to ETH base address + maximum ETH register address offset

Table 3-18 Peripheral Region address range

The OS callbacks that are called by the Ethernet Driver are provided by the Operating System component. Please refer to the Technical Reference [9] of the OS component for further information.

3.14.5.2 Non-Privileged Access by Configuration of a Peripheral Protection Unit

A Peripheral Protection Unit (PPU) which is existing on certain platforms can be configured to permit access to registers which normally require Privileged access mode also for a non-privileged access in User mode. These access rights can be configured on certain platforms individually per peripheral unit, e.g. for a whole Ethernet MAC unit. For further

details see e.g. section ‘Peripheral Protection Unit’ of the related hardware manual of the used derivative.

3.14.6 Receive Buffer Segmentation

Some Ethernet MACs provide the ability to receive one frame using multiple descriptors connected to small buffer. When this feature is available and enabled the configuration allows configuring segment sizes different to the maximum frame size, which can be received. The MAC uses as much descriptors per frame as necessary to receive the whole frame.

The implementation of this mechanism is transparent for the upper layers.

Enabling Receive Buffer Segmentation can be activated for a more efficient usage of RAM, which is needed in systems which need to receive large frames only in rare cases, but mainly receive small frames. Instead of having a pool of large buffers available which cause a lot of wasted memory for small frames, the usage of memory scales with the amount of used segments per frame.

Considering an example of three frames with a size of 128 Bytes and one frame with a size of 1500 Bytes, without Receive Buffer Segmentation 4 x 1500 = 6000 Bytes of receive buffers are needed. Using Receive Buffer Segmentation only 15 x 128 = 1920 Bytes of receive buffers are needed.

3.14.7 Runtime Environment INTEGRITY

The INTEGRITY operating system is a real-time operating system (RTOS) developed and provided by Green Hills Software.

The rough concept of the OS is to define virtual address spaces (besides the kernel address space which is privileged) that are separated from each other. This allows to run critical software components besides non-critical software components and ensure that misbehavior of the non-critical ones cannot cause a failure in the critical ones. For more information about the operation system refer to the Green Hills Software website.



Caution
The driver was implemented for and tested with INTEGRITY version 11.0.4.
Other INTEGRITY versions are not supported.

The Vector Ethernet controller driver is designed to run within the INTEGRITY AUTOSAR environment. This environment is defined as a virtual address space where INTEGRITY provides AUTOSAR OS compatible interfaces. An AUTOSAR compliant stack can interface with these and therefore execute within such a virtual address space.

The driver itself is composed of two parts. The actual driver that runs in the AUTOSAR environment and an IODevice running in the kernel address space of INTEGRITY.

The IODevice allocates and provides INTEGRITY specific resources and hardware access to the driver part running in an AUTOSAR environment but doesn’t implement any Ethernet controller specific functionality. This is all included in the driver part.

Due to this separation only the INTEGRITY AUTOSAR environment can access the Ethernet controller and in consequence communicate over Ethernet. If other virtual address spaces shall be able to communicate over Ethernet they must provide/collect the respective information to the INTEGRITY AUTOSAR environment and send/receive Ethernet frames with the help of the MICROSAR Ethernet stack.



Caution
The current implementation of the MICROSAR driver is classified as QM acc. to ISO 26262.
Therefore, no measures and activities have been executed to support any Automotive Safety Integrity Level (ASIL A-D) acc. to ISO 26262.
As a matter of fact, Vector is not responsible for any malfunction and further risks, e.g. corrupting kernel operation, by using this driver in an ASIL environment. This currently limits the usage to serial production projects that don’t need to be qualified according to an ISO 26262 ASIL level.



Caution
Currently only polling mode operation is supported by the driver when running in INTEGRITY environment.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR Ethernet Driver into an application environment of an ECU.

4.1 Embedded Implementation

The delivery of the Ethernet Driver consists out of the following files.

If a driver needs additional source files the list of the additions can be found in [6].

File Name	Description	Integration Tasks
Eth_30_<Driver>.c	Static source file that contains public API implementation.	-
Eth_30_<Driver>.h	Static header file that contains public API declaration.	-
Eth_30_<Driver>_HwAccess_Int.h	Static header file that contains the hardware access internal declaration and definitions.	-
Eth_30_<Driver>_Int.h	Static header file that contains the general internal declaration and definitions.	-
Eth_30_<Driver>_IrqHandler.c	Static source file that contains interrupt handler implementation.	-
Eth_30_<Driver>_IrqHandler_Int.h	Static header file that contains internal interrupt handler declaration and definitions.	-
Eth_30_<Driver>_LL.c	Static source file that contains hardware dependent implementation.	-
Eth_30_<Driver>_LL.h	Static header file that contains hardware specific declaration and definitions.	-
Eth_30_<Driver>_LL_Int.h	Static header file that contains hardware specific internal declaration and definitions.	-
Eth_30_<Driver>_LL_IrqHandler.c	Static source file that contains hardware specific interrupt handler implementation.	-
Eth_30_<Driver>_LL_IrqHandler.h	Static header file that contains hardware specific interrupt handler declaration and definitions.	-
Eth_30_<Driver>_LL_Regs.h	Static header file that contains hardware specific register definitions.	-

File Name	Description	Integration Tasks
Eth_30_<Driver>_LL_TimeSync.c[1]	Static source file that contains hardware specific time synchronization implementation.	-
Eth_30_<Driver>_LL_TimeSync.h[1]	Static header file that contains hardware specific time synchronization declaration and definitions.	-
Eth_30_<Driver>_LL_TrafficHandling.h[2]	Static header file that contains hardware specific FQTSS declaration and definitions.	-
Eth_30_<Driver>_LL_Types.h	Static header file that contains hardware specific types.	-
Eth_30_<Driver>_TimeSync.c[1]	Static source file that contains general time synchronization implementation.	-
Eth_30_<Driver>_TimeSync.h[1]	Static header file that contains general time synchronization declaration and definitions.	-
Eth_30_<Driver>_TimeSync_Int.h[1]	Static header file that contains internal time synchronization declaration and definitions.	-
Eth_30_<Driver>_TrafficHandling.c[2]	Static source file that contains general FQTSS implementation.	-
Eth_30_<Driver>_TrafficHandling.h[2]	Static header file that contains general FQTSS declaration and definitions.	-
Eth_30_<Driver>_TrafficHandling_Int.h[2]	Static header file that contains internal FQTSS declaration and definitions.	-
Eth_30_<Driver>_IODev.c[3]	Static source file that contains the INTEGRITY IODevice implementation.	-
Eth_30_<Driver>_IODev.h[3]	Static header file that contains the INTEGRITY IODevice declaration and definitions.	-
Eth_30_<Driver>_IODev_Types.h[3]	Static header file that contains the INTEGRITY IODevice types.	-
Eth_30_<Driver>_IODev_AsrIfc.h[3]	Static header file that contains the AUTOSAR environment to interface with the IODevice.	-

[1] Only available, if the feature “Time Synchronization” is available for the delivered driver / hardware.
[2] Only available, if the feature „FQTSS (Traffic Shaping)” is available for the delivered driver / hardware.
[3] Only available, if the driver is delivered to be used in a Greenhills INTEGRITY environment.

File Name	Description	Integration Tasks
Eth_30_<Driver>_LL_IODev.h[3]	Static header file that contains the hardware dependent declaration and definitions for the IODevice.	-
Eth_30_<Driver>_Types.h	Static header file that contains public types.	-
Eth_30_<Driver>_Cfg.h	Generated header file that contains the pre-compile time configuration.	-
Eth_30_<Driver>_Lcfg.c	Generated source file that contains the Link-time parameter configuration.	-
Eth_30_<Driver>_Lcfg.h	Generated header file that contains the Link-time parameter declaration and config access definitions.	-
Eth_30_<Driver>_GenTypes.h	Generated header file that contains the types of the generated data.	-
Eth_30_<Driver>_Irq.c	Generated source file that contains the interrupt service routine implementation.	-
Eth_30_<Driver>_MemMap.h	Generated header file that contains the memory mapping sections for buffers and descriptors.	-
Eth_30_<Driver>_IODev_Cfg.h[3]	Generated header file that contains the INTEGRITY IODevice configuration declaration.	-
Eth_30_<Driver>_IODev_Cfg.h[3]	Generated source file that contains the INTEGRITY IODevice configuration implementation.	-
Eth_30_<Driver>_ASR_generated.int[3]	Generated INTEGRITY integrate file containing INTEGRITY resources to be allocated through build time within the AUTOSAR environment virtual address space	-
Eth_30_<Driver>_kernel_generated.int[3]	Generated INTEGRITY integrate file containing INTEGRITY resources to be allocated during build time within the kernel address space.	-

File Name	Description	Integration Tasks
_Eth_30_<Driver>_Cbk.h	**Template file** that must be adapted by the user. This file contains the definition for Appl_Eth_30_<Driver>_MemoryBarrier.	Rename template file to Eth_30_<Driver>_Cbk.h. Adapt the file. For further instructions refer to the description within the file. For affected drivers, additional information is presented in [6].

Table 4-1 List of delivered files

4.2 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

Table 4-2 contains the memory section names and the compiler abstraction definitions which are defined for the Ethernet Driver and illustrates their assignment among each other. Each section start is prefixed with `ETH_30_<DRIVER>_START` and the section end with `ETH_30_<DRIVER>_STOP`, e.g. `ETH_30_<DRIVER>_START_SEC_CONST_UNSPECIFIED`.

Memory Mapping Sections \ Compiler Abstraction Definitions	ETH_30_<DRIVER>_CONST	ETH_30_<DRIVER>_VAR	ETH_30_<DRIVER>_CODE
…_SEC_CONST_UNSPECIFIED	■		
…_SEC_CONST_32BIT	■		
…_SEC_CONST_16BIT	■		
…_SEC_CONST_8BIT	■		
…_SEC_VAR_NOINIT_UNSPECIFIED		■	
…_SEC_VAR_NOINIT_32BIT		■	
…_SEC_VAR_NOINIT_16BIT		■	
…_SEC_VAR_NOINIT_8BIT		■	
…_SEC_TX_BUFFER_VAR_NOINIT_UNSPECIFIED		■	
…_SEC_RX_BUFFER_VAR_NOINIT_UNSPECIFIED		■	

…_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED		■	
…_SEC_RX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED		■	
…_SEC_DESCRIPTOR_BASE_ADDRESS_TABLE _VAR_NOINIT_UNSPECIFIED		■	
…_SEC_VAR_NOINIT_BOOLEAN		■	
…_SEC_CODE			■

Table 4-2 Compiler abstraction and memory mapping

4.3 Exclusive Areas

This section describes the exclusive areas utilized by the Ethernet Driver to ensure data consistency. The general exclusive areas used by all drivers are listed in the following table. For exclusive areas needed by a specific driver please see [6].



Note

For better readability the prefix `ETH_30_<DRIVER>_EXCLUSIVE_AREA` of the exclusive area names was removed and must be considered during integration.

When using DaVinci Configurator PRO and the respective MICROSAR stack there is no need to configure these areas manually. It is done by the tool automatically.

Exclusive Area	Descriptor
MII	This exclusive area ensures a consistent and exclusive access to the MDIO interface of the Ethernet Controller.
DATA	This exclusive are ensures the state and data consistency during reception and transmission of an Ethernet frame.
RXTX_STATS	This exclusive area ensures data consistency during reading RX and TX statistic values.

Table 4-3 Exclusive areas

4.4 Memory Mapping

The Ethernet buffers and descriptors must be mapped with the AUTOSAR memory mapping feature. To provide flexibility with regards to the used memory areas, the memory sections differ between the Rx and Tx sections and an additional section for the descriptor base address table. The mappable sections are:

`ETH_30_<DRIVER>_START_SEC_RX_BUFFER_VAR_NOINIT_8BIT`,

`ETH_30_<DRIVER>_START_SEC_TX_BUFFER_VAR_NOINIT_8BIT`

`ETH_30_<DRIVER>_START_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED`,

`ETH_30_<DRIVER>_START_SEC_RX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED` and

`ETH_30_<DRIVER>_START_SEC_DESCRIPTOR_BASE_ADDRESS_TABLE_VAR_NOINIT_UNSPECIFIED`

The definitions for these sections are generated into the `Eth_30_<Driver>_MemMap.h` file.

Each section must be placed and aligned according to the hardware specific restrictions defined by the Ethernet controller peripheral managed by this DrvEth. Refer to [6] for further information on the hardware specifics. If the placement and/or alignment of the buffers and descriptors is not done correctly, the driver will most likely be inoperable.

In the following sections some examples can be found that describe how the mapping can be done for the most common compilers. The sections `EthRamTxDesc` (used for the Tx descriptors) and `EthRamTxBuf` (used for the Tx buffers) must be aligned according to the alignment restrictions given in [6] in the linker configuration. The same applies for the Rx path and descriptor base address table. If no specific mappings or alignment differences between the Rx, Tx and base address table exist, they can also be mapped into the same section, e.g. `EthRamBuf`. Please refer to the hardware manual for the used platform for more information what must be done to integrate this driver.

4.4.1 Windriver DiabData Compiler



Example

```
#ifdef ETH_30_<DRIVER>_START_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# undef ETH_30_<DRIVER>_START_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# define START_SEC_VAR_NOINIT_UNSPECIFIED
# pragma section DATA „.EthRamTxDesc“ „.EthRamTxDesc“
#endif
#ifdef ETH_30_<DRIVER>_STOP_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# undef ETH_30_<DRIVER>_STOP_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# pragma section DATA
# define STOP_SEC_VAR
#endif

#ifdef ETH_30_<DRIVER>_START_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# undef ETH_30_<DRIVER>_START_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# define START_SEC_VAR_NOINIT_UNSPECIFIED
# pragma section DATA „.EthRamTxBuf“ „.EthRamTxBuf“
#endif
#ifdef ETH_30_<DRIVER>_STOP_SEC_TX_BUFFER_VAR_NOINIT_8BIT
```

```
# undef ETH_30_<DRIVER>_STOP_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# pragma section DATA
# define STOP_SEC_VAR
#endif
```

4.4.2 Green Hills Compiler



Example

```
#ifdef ETH_30_<DRIVER>_START_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# undef ETH_30_<DRIVER>_START_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# define START_SEC_VAR_NOINIT_UNSPECIFIED
# pragma ghs section bss=“.EthRamTxDesc“
#endif
#ifdef ETH_30_<DRIVER>_STOP_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# undef ETH_30_<DRIVER>_STOP_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# pragma ghs section bss=default
# define STOP_SEC_VAR
#endif

#ifdef ETH_30_<DRIVER>_START_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# undef ETH_30_<DRIVER>_START_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# define START_SEC_VAR_NOINIT_UNSPECIFIED
# pragma ghs section bss=“.EthRamTxBuf“
#endif
#ifdef ETH_30_<DRIVER>_STOP_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# undef ETH_30_<DRIVER>_STOP_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# pragma ghs section bss=default
# define STOP_SEC_VAR
#endif
```

4.4.3 Altium TASKING Compiler



Example

```
#ifdef ETH_30_<DRIVER>_START_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# undef ETH_30_<DRIVER>_START_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED #
pragma section all “EthRamTxDesc”
# define START_SEC_VAR_NOINIT_UNSPECIFIED
#endif
#ifdef ETH_30_<DRIVER>_STOP_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# undef ETH_30_<DRIVER>_STOP_SEC_TX_DESCRIPTOR_VAR_NOINIT_UNSPECIFIED
# pragma section all restore
# define STOP_SEC_VAR
#endif

#ifdef ETH_30_<DRIVER>_START_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# undef ETH_30_<DRIVER>_START_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# pragma section all "EthRamTxBuf"
# define START_SEC_VAR_NOINIT_UNSPECIFIED
#endif
#ifdef ETH_30_<DRIVER>_STOP_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# undef ETH_30_<DRIVER>_STOP_SEC_TX_BUFFER_VAR_NOINIT_8BIT
# pragma section all restore
```

```
# define STOP_SEC_VAR
#endif
```

4.5 Interrupts

The Ethernet driver supports the configuration of an interrupt mapping.

In case the interrupts are enabled and an OS is running, there are two different interrupt categories, provided by the OS. The Ethernet driver supports both Category 1 and Category 2 interrupts. Additionally, the interrupt operation without an OS is possible.

Caution
When using Category 1 interrupts it is platform dependent how the stack for the interrupt is allocated. Because the stack size needed by an Ethernet interrupt is usually greater than for other interrupts the platform specific behavior must be considered. Otherwise this could lead to stack overflows.

For detailed information about how the stack for a Category 1 interrupt is allocated on the used platform please refer to the technical reference of the used AUTOSAR OS.

For usage of the interrupt service routines and their configuration please refer to [6].

Due to support for different derivatives a generic interrupt service routine configuration is provided.

4.6 Runtime environment INTEGRITY

INTEGRITY RTOS is setup with the help of the MULT IDE. For a detailed description please refer to the manuals of your INTEGRITY software package.

Since the driver is composed of two parts, these must be integrated into the INTEGRITY project each for themselves.

Caution
The INTEGRITY project must be defined as a monolith system. Dynamic download systems are not supported.

Caution
It must be ensured that the BSP is configured to not include any Ethernet controller driver that accesses the Ethernet controller the MICROSAR driver shall control.

4.6.1 MICROSAR driver

The MICROSAR driver doesn't need special integration. It is configured with the DaVinci Configurator PRO and compiled and linked with the whole MICROSAR stack (plus the INTEGRITY specific AUTOSAR components) to a library that can be linked to the AUTOSAR environment virtual address space within the INTEGRITY project.

The only manual step that must be taken is to include the following folder to the include directory list so the MICROSAR driver is able to interface to the IODevice through the `Eth_30_<Driver>_IODev_AsrIfc.h` header file.

`<VectorSIP>/BSW/Eth_30_<Driver>/GHS_INTEGRITY/AsrWrapperIfc`

The AUTOSAR environment virtual address space project could then look like shown in the following figure, where EthTestApp.lib is the library containing the MICROSAR stack.

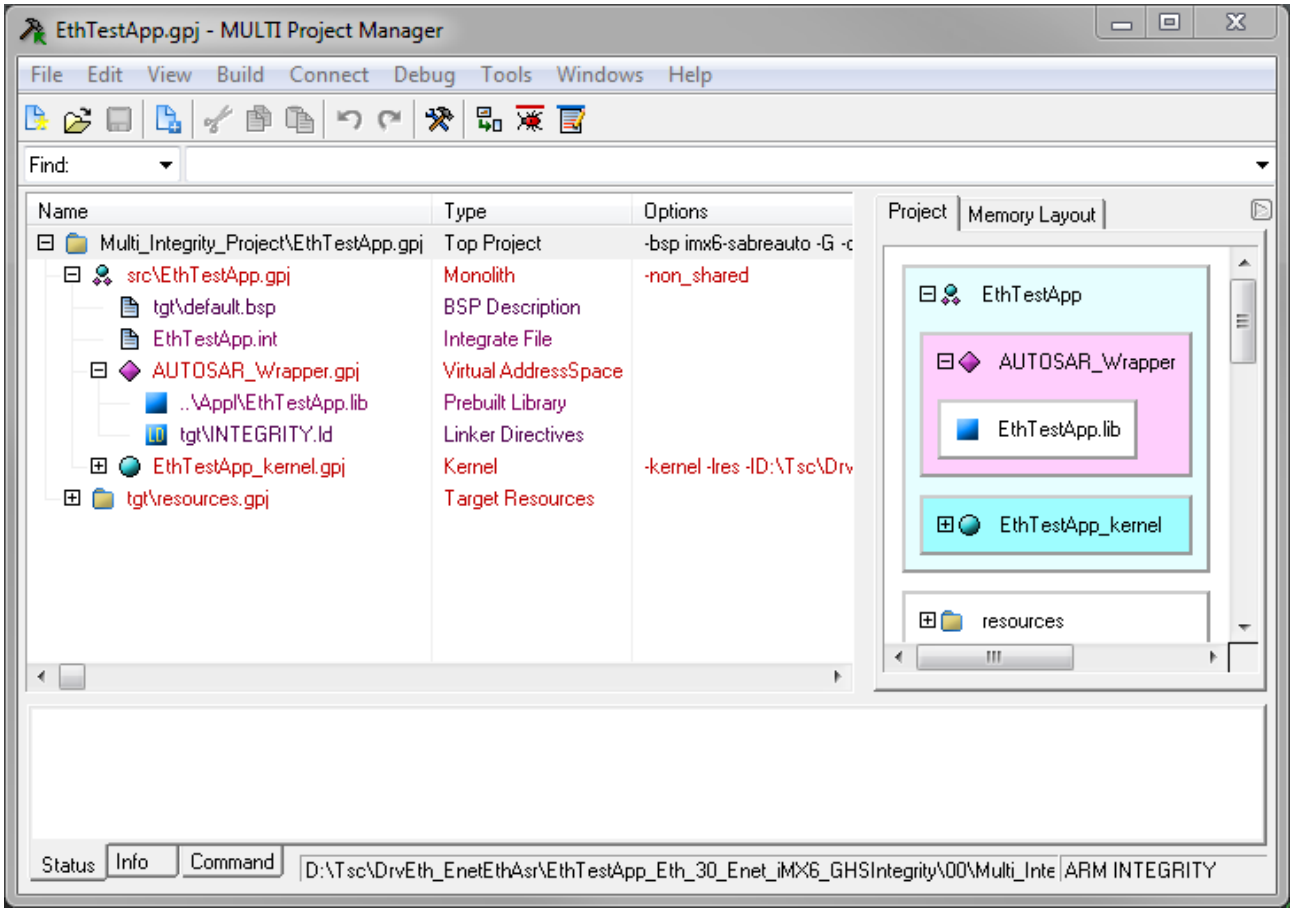


Figure 4-1 INTEGRITY AUTOSAR environment project setup

4.6.2 IODevice

The kernel part of the driver, the IODevice, must be integrated manually into the INTEGRITY project.

It is composed of two types of files. C-files containing the implementation of the IODevice and *.int files provided to the Green Hills Integrate tool, that allows to define and allocate INTEGRITY resources during build time of the INTEGRITY project.



Note
The driver relies on the functionality that is provided by the ResourceManager of INTEGRITY. Therefore, the ResourceManager must be activated for the kernel address space project.

Source file integration

IODevice source files must be manually added to the kernel address space.

The files that must be added as source files are:

`<VectorSIP>/BSW/Eth_30_<Driver>/GHS_INTEGRITY/Eth_30_<Driver>_IODev.c`

`<DaVinciProject>/Appl/Source/Eth_30_<Driver>_IODev_Cfg.c`

Additionally, the following folders must be added as include path to the kernel address space project:

`<VectorSIP>/BSW/Eth_30_<Driver>/GHS_INTEGRITY`

`<VectorSIP>/BSW/Eth_30_<Driver>/GHS_INTEGRITY/AsrWrapperIfc`

`<DaVinciProject>/Appl/Source`

The kernel address space could then look like shown in the following figure, where the noted files are added.

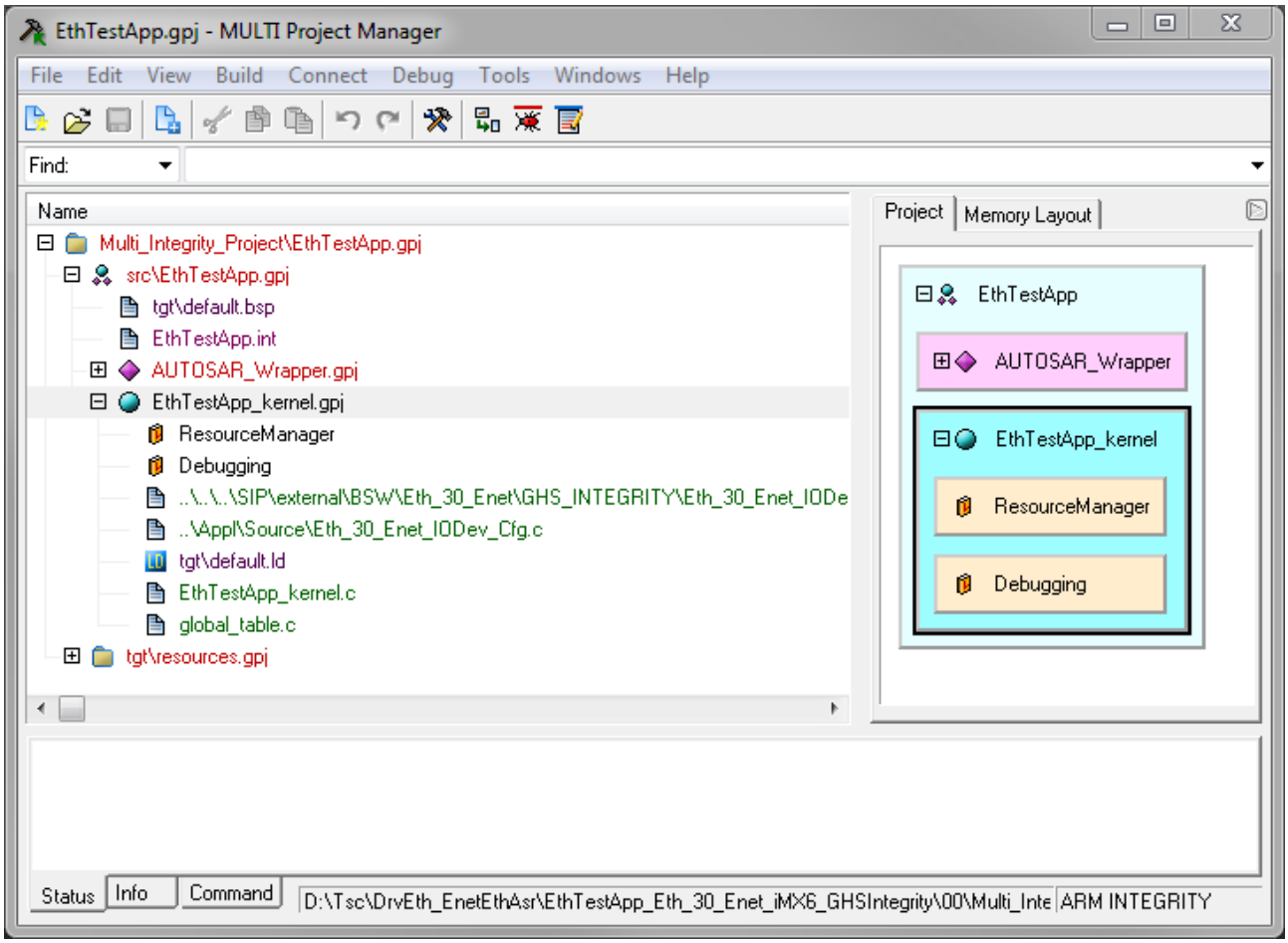


Figure 4-2 INTEGRITY kernel project setup

Integrate file integration

Each INTEGRITY project also contains a *.int file that is provided to the Green Hills Integrate tool. The file itself is named <MULTIProjectName>.int. This file must be adapted to include following files (and the <DaVinciProjectName>.int file generated by the INTEGRITY components included in the DaVinci Configurator PRO project).

```
<DaVinciProject>/Appl/Source/Eth_30_<Driver>_ASR_generated.int
<DaVinciProject>/Appl/Source/Eth_30_<Driver>_kernel_generated.int
```

The <MULTIProjectName>.int file could then look like shown in the following figure, where the noted files are added.

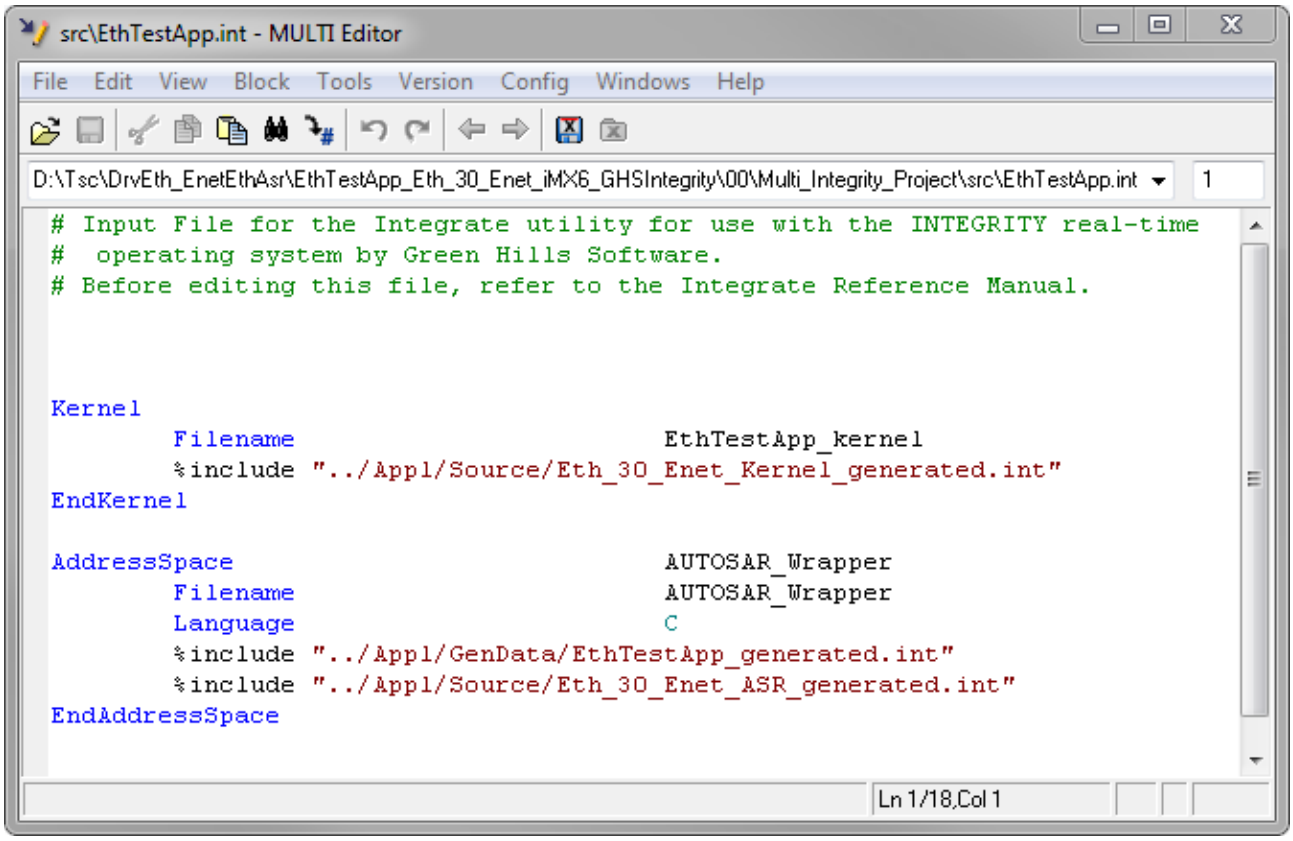


Figure 4-3 INTEGRITY integrate file setup

5 API Description

For an interfaces overview please see Figure 2-1.

5.1 Type Definitions

The types defined by the Ethernet Driver are described in this chapter.

Type Name	C-Type	Description	Value Range
Eth_ConfigType	void	Ethernet controller configuration pointer type	Pointer Pointer to configuration
Eth_ReturnType	uint8	Return type of Eth_30_<Driver>_ReadMii() and Eth_30_<Driver>_WriteMii() APIs	ETH_OK Success
			ETH_E_NOT_OK General failure
			ETH_E_NO_ACCESS Ethernet hardware access failure
Eth_ModeType	uint8	Defines all possible controller modes	ETH_MODE_DOWN Controller inactive
			ETH_MODE_ACTIVE Normal operation mode
			ETH_TX_STATE_NOT_TRANSMITTED Frame not yet transmitted
Eth_FrameType	uint16	Ethernet Frame Type	0x0000 - 0xFFFF Any Ethernet frame type
Eth_DataType	uint32	Defines the Ethernet data type	0x00000000 - 0xFFFFFFFF User data
Eth_RxStatusType	uint8	Out parameter in Eth_30_<Driver>_Receive() to indicate that a frame has been received and if so, whether more frames are available or frames got lost	ETH_RECEIVED Frame received, no further frames available
			ETH_NOT_RECEIVED Frame received, no further frames available
			ETH_RECEIVED_MORE_DATA_AVAILABLE Frame received, more frames available
			ETH_RECEIVED_FRAMES_LOST Frame received, some frames lost
Eth_FilterActionType	uint8	Describes the action to	ETH_ADD_TO_FILTER

Type Name	C-Type	Description	Value Range
		be taken for the MAC address given to API Eth_30_<Driver>_UpdatePhysAddrFilter()	Add MAC to the filter, meaning allow reception
			ETH_REMOVE_FROM_FILTER Remove MAC from the filter, meaning reception is blocked in the lower layer
Eth_StateType	uint8	Defines all possible controller Driver states	ETH_STATE_UNINIT Ethernet Controller Driver not initialized
			ETH_STATE_INIT Ethernet Controller Driver initialized
			ETH_STATE_ACTIVE Ethernet Controller Driver enabled
			ETH_STATE_DOWN Ethernet Controller Driver disabled
Eth_TimestampQualityType	uint8	Defines the quality of a timestamp	ETH_TIMESTAMP_VALID Returned timestamp is valid
			ETH_TIMESTAMP_INVALID Returned timestamp is invalid, e.g. because of an hardware error
			ETH_TIMESTAMP_UNCERTAIN Timestamp validity is uncertain. This value is not used by the Tricore Ethernet Driver!
Eth_TimediffType	sint32	Defines an offset to correct the time offset of the Ethernet controllers clock	-2,147,483,648 - 2,147,483,647 [ns]
Eth_RateRatioType	float64	Defines a ratio rate that is used to correct the time drift of the Ethernet controllers clock	0.0 - 2.0
Eth_MeasurementIdxType	uint8	Defines the measurement index, i.e. the type of data requested to be retrieved and/or reset	ETH_MEAS_ALL Reset all data
			ETH_MEAS_VENDOR_SPECIFIC_DROP_INSUFF_TX_BUFFER Retrieve or Reset the amount of unsuccessful Tx buffer provision requests due to lacking Tx buffer resources
			ETH_MEAS_VENDOR_SPECIFI

Type Name	C-Type	Description	Value Range
			C_DROP_INSUFF_RX_BUFFER Retrieve or Reset the amount of Rx frame drop events due to lacking Rx buffer resources ETH_MEAS_VENDOR_SPECIFIC_WARN_INSUFF_RX_BUFFER Retrieve or Reset the amount of times the Rx buffers are all occupied during a frame reception event

Table 5-1 Type definitions

5.2 Structure Definitions

5.2.1 Eth_RxStatsType

Defines a list of Ethernet statistic counters for reception which is also described in [2] and IETF RFC 2819 MIB.

The highest possible value of each counter value range (e.g. value 2^{32}) has the special meaning of ‘counter not available’.

The second highest value of each counter value range (e.g. value 2^{32}-1) has the special meaning of ‘counter overflow’.

Struct Element Name	C-Type	Description	Value Range
RxStatsDropEvents	uint32	The total number of events in which packets were dropped due to lack of ressources.	[0 ; 2^{32}]
RxStatsOctets	uint32	The total number of octets of data (including those in bad packets) received on the network (excluding framing bits but including FCS octets).	[0 ; 2^{32}]
RxStatsPkts	uint32	The total number of packets (including bad packets, broadcast packets and multicast packets) received.	[0 ; 2^{32}]
RxStatsBroadcastPkts	uint32	The total number of good packets received that were directed to the broadcast address.	[0 ; 2^{32}]
RxStatsMulticastPkts	uint32	The total number of good packets received that were directed to a multicast address.	[0 ; 2^{32}]
RxStatsCrcAlignErrors	uint32	The total number of packets received that had a length between 64 and 1518 octets	[0 ; 2^{32}]

Struct Element Name	C-Type	Description	Value Range
		that had either a bad Frame Check Sequence (FCS) with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error).	
RxStatsUndersizePkts	uint32	The total number of packets received that were less than 64 octets long (excluding framing bits, but including FCS octets) and were otherwise well formed.	[0 ; 2^{32}]
RxStatsOversizePkts	uint32	The total number of packets received that were longer than 1518 octets long (excluding framing bits, but including FCS octets) and were otherwise well formed.	[0 ; 2^{32}]
RxStatsFragments	uint32	The total number of packets received that were less than 64 octets in length (excluding framing bits but including FCS octets) and had either a bad FCS with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error).	[0 ; 2^{32}]
RxStatsJabbers	uint32	The total number of packets received that were longer than 1518 octets in length and had either a bad FCS with an integral number of octets (FCS Error) or a bad FCS with a non-integral number of octets (Alignment Error).	[0 ; 2^{32}]
RxStatsCollisions	uint32	The best estimate of the total number of collisions on this Ethernet segment.	[0 ; 2^{32}]
RxStatsPkts64Octets	uint32	The total number of packets (including bad packets) received that were 64 octets in length.	[0 ; 2^{32}]
RxStatsPkts65to127Octets	uint32	The total number of packets (including bad packets) received that were between 65 and 127 octets in length.	[0 ; 2^{32}]
RxStatsPkts128to255Octets	uint32	The total number of packets (including bad packets) received that were between 128 and 255 octets in length.	[0 ; 2^{32}]

Struct Element Name	C-Type	Description	Value Range
RxStatsPkts256to511Octets	uint32	The total number of packets (including bad packets) received that were between 256 and 511 octets in length.	[0 ; 2^{32}]
RxStatsPkts512to1023Octets	uint32	The total number of packets (including bad packets) received that were between 512 and 1023 octets in length.	[0 ; 2^{32}]
RxStatsPkts1024to1518Octets	uint32	The total number of packets (including bad packets) received that were between 1024 and 1518 octets in length.	[0 ; 2^{32}]
RxUnicastFrames	uint32	The number of subnetwork-unicast packets delivered to a higher level protocol.	[0 ; 2^{32}]

Table 5-2 Eth_RxStatsType Structure

5.2.2 Eth_TxStatsType

Defines a list of Ethernet statistic counters for transmission, which is also described in [2] and IETF RFC 2819 MIB.

The highest possible value of each counter value range (e.g. value 2^{32}) has the special meaning of ‘counter not available’.

The second highest value of each counter value range (e.g. value 2^{32}-1) has the special meaning of ‘counter overflow’.

Struct Element Name	C-Type	Description	Value Range
TxNumberOfOctets	uint32	The total number of octets transmitted out of the interface, including framing characters.	[0 ; 2^{32}]
RxNUcastPkts	uint32	The total number of packets that higher level protocols requested to be transmitted to a non-unicast (i.e., a subnetwork-broadcast or subnetwort-multicast) address, including those that were discarded or not sent.	[0 ; 2^{32}]
TxUniCastPkts	uint32	The total number of packets that higher level protocols requested to be transmitted to a subnetwork-unicast address, including those that were discarded or not sent.	[0 ; 2^{32}]

Table 5-3 Eth_TxStatsType Structure

5.2.3 Eth_TimeStampType

Defines a timestamp, according to IEEE 1588-2008 (PTP version 2)

Struct Element Name	C-Type	Description	Value Range
nanoseconds	uint32	Nanoseconds part of time	[0 ; 999,999,999] [0x00 ; 0x3B9A_C9ff]
seconds	uint32	32 LSB of the 48 Bit seconds part of time	[0 ; 2^{32}]
secondsHi	uint16	16 MSB of the 48 Bit seconds part of time	[0 ; 2^{16}]

Table 5-4 Eth_TimeStampType Structure

5.3 API Table

5.3.1 AUTOSAR API

5.3.1.1 Eth_30_Core_InitMemory

Prototype	
`void` **`Eth_30_Core_InitMemory`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initializes *_INIT_*-variables.	
Particularities and Limitations	
Module is uninitialized Service to initialize module global variables at power up. This function initializes the variables in *_INIT_* sections. Used in case they are not initialized by the startup code.	
Call context	
> TASK > This function is synchronous > This function is non-Reentrant	

Table 5-5 Eth_30_Core_InitMemory

5.3.1.2 Eth_30_Core_Init

Prototype	
`void Eth_30_Core_Init (const Eth_ConfigType *CfgPtr)`	
Parameter	
CfgPtr [in]	Pointer to post-build configuration or null pointer
Return code	
void	none
Functional Description	
Initializes the module.	
Particularities and Limitations	
Module's *_INIT*_-variables are initialized either by Eth_30_Core_InitMemory() or startup code Function initializes the module Eth_30_Core. It initializes all variables and sets the module state to initialized.	
Call context	
> TASK > This function is synchronous > This function is non-reentrant	

Table 5-6 Eth_30_Core_Init

5.3.1.3 Eth_30_Core_ControllerInit

Prototype	
`Std_ReturnType Eth_30_Core_ControllerInit (uint8 CtrlIdx, uint8 CfgIdx)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
CfgIdx [in]	Identifier of the configuration (only 0 allowed)
Return code	
Std_ReturnType	E_NOT_OK - Initialization of Ethernet controller failed
	E_OK - Ethernet controller initialized
Functional Description	
Initializes an Ethernet controller.	
Particularities and Limitations	
Module is initialized Function initializes an Ethernet controller and the related variables, so it is possible to set it in operation afterwards.	
Call context	
> TASK > This function is synchronous	

> This function is reentrant

Table 5-7 Eth_30_Core_ControllerInit

5.3.1.4 Eth_30_Core_SetControllerMode

Prototype	
`Std_ReturnType` **`Eth_30_Core_SetControllerMode`** `(uint8 CtrlIdx, Eth_ModeType CtrlMode)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
CtrlMode [in]	Operation mode that shall be applied:
	ETH_MODE_DOWN - Ethernet controller shall be turned off
	ETH_MODE_ACTIVE - Ethernet controller shall be turned on
Return code	
Std_ReturnType	E_NOT_OK - Operation mode couldn't be applied
	E_OK - Operation mode successfully applied
Functional Description	
Sets the operation mode of an Ethernet controller.	
Particularities and Limitations	
Ethernet controller is initialized Function sets the operation mode of the Ethernet controller, so it is either turned off (no frame reception and transmission) or turned on (frames can be transmitted and received).	
Call context	
> TASK > This function is synchronous > This function is reentrant	

Table 5-8 Eth_30_Core_SetControllerMode

5.3.1.5 Eth_30_Core_GetControllerMode

Prototype	
`Std_ReturnType` **`Eth_30_Core_GetControllerMode`** `(uint8 CtrlIdx, Eth_ModeType *CtrlModePtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
CtrlModePtr [out]	Operation mode retrieved
Return code	
Std_ReturnType	E_NOT_OK - Retrieval of operation mode failed

	E_OK - Operation mode successfully retrieved
Functional Description	
Retrieves the current operation mode of an Ethernet controller.	
Particularities and Limitations	
Ethernet controller is initialized -	
Call context	
> TASK > This function is synchronous > This function is reentrant	

Table 5-9 Eth_30_Core_GetControllerMode

5.3.1.6 Eth_30_Core_GetPhysAddr

Prototype	
`void Eth_30_Core_GetPhysAddr (uint8 CtrlIdx, uint8 *PhysAddrPtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
PhysAddrPtr [out]	Buffer of at least 6 bytes to pass the MAC address
Return code	
void	none
Functional Description	
Retrieves the currently active MAC address of an Ethernet controller.	
Particularities and Limitations	
Module is initialized -	
Call context	
> TASK > This function is synchronous > This function is reentrant	

Table 5-10 Eth_30_Core_GetPhysAddr

5.3.1.7 Eth_30_Core_SetPhysAddr

Prototype	
`void Eth_30_Core_SetPhysAddr (uint8 CtrlIdx, const uint8 *PhysAddrPtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller

PhysAddrPtr [in]	Buffer holding the MAC address that shall be applied
Return code	
void	none
Functional Description	
Sets the MAC address of an Ethernet controller.	
Particularities and Limitations	
Module is initialized Function sets the MAC address of an Ethernet controller. Dependent on the configuration of the "Write MAC address" feature the change is persisted in non-volatile RAM and available after a power- cycle of the MCU.	
Call context	
> TASK > This function is synchronous > This function is reentrant	

Table 5-11 Eth_30_Core_SetPhysAddr

5.3.1.8 Eth_30_Core_UpdatePhysAddrFilter

Prototype	
`Std_ReturnType Eth_30_Core_UpdatePhysAddrFilter (uint8 CtrlIdx, const uint8 *PhysAddrPtr, Eth_FilterActionType Action)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
PhysAddrPtr [in]	Buffer holding the MAC address the filter shall be adapted for
Eth_FilterActionType [in]	Action that shall be applied for the filter:
	ETH_REMOVE_FROM_FILTER - MAC address shall be blocked
	ETH_ADD_TO_FILTER - MAC address shall be allowed
Return code	
Std_ReturnType	E_NOT_OK - Filter modification failed
	E_OK - Filter successfully updated
Functional Description	
Updates the reception MAC address filter of an Ethernet controller.	
Particularities and Limitations	
Ethernet controller is initialized Functions allows to add or remove MAC address from the reception filter of the Ethernet controller, so Ethernet frames addressed to the respective MAC address can be received or will be blocked from reception.	
Call context	
> TASK > This function is synchronous	

> This function is reentrant

Table 5-12 Eth_30_Core_UpdatePhysAddrFilter

5.3.1.9 Eth_30_Core_WriteMii

Prototype	
`Eth_ReturnType` **`Eth_30_Core_WriteMii`** `(uint8 CtrlIdx, uint8 TrcvIdx, uint8 RegIdx, uint16 RegVal)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
TrcvIdx [in]	Address of the counter part on MDIO interface (MII address)
RegIdx [in]	Address of the register that shall be written
RegVal [in]	Value that shall be written to the register
Return code	
Eth_ReturnType	ETH_E_NOT_OK - Service call failed due to invalid module state or function parameters
	ETH_E_NO_ACCESS - Access to the MDIO interface timed out
	ETH_OK - Write command was triggered successfully
Functional Description	
Triggers a write command on the MDIO interface of an Ethernet controller.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE Function triggers a write command on the MDIO interface according to clause 22 of the IEEE specification.	
Call context	
> ANY > This function is synchronous > This function is reentrant for different MDIO interfaces	

Table 5-13 Eth_30_Core_WriteMii

5.3.1.10 Eth_30_Core_ReadMii

Prototype	
`Eth_ReturnType` **`Eth_30_Core_ReadMii`** `(uint8 CtrlIdx, uint8 TrcvIdx, uint8 RegIdx, uint16 *RegValPtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
TrcvIdx [in]	Address of the counter part on MDIO interface (MII address)
RegIdx [in]	Address of the register that shall be read
RegValPtr [out]	Buffer to store the result of the read command

Return code	
Eth_ReturnType	ETH_E_NOT_OK - Service call failed due to invalid module state or function parameters
	ETH_E_NO_ACCESS - Access to the MDIO interface timed out
	ETH_OK -
Functional Description	
Triggers a read command on the MDIO interface of an Ethernet controller an provides the result.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE Function triggers a read command on the MDIO interface according to clause 22 of the IEEE specification and provides the result of the read.	
Call context	
> ANY > This function is synchronous > This function is reentrant for different MDIO interfaces	

Table 5-14 Eth_30_Core_ReadMii

5.3.1.11 Eth_30_Core_GetCounterState

Prototype
`Std_ReturnType` **`Eth_30_Core_GetCounterState`** `(uint8 CtrlIdx, uint16 CtrOffs, uint32 *CtrValPtr)`
Parameter
CtrlIdx [in] — Identifier of the Ethernet controller
CtrOffs [in] — Offset of the counter into the Ethernet statistics counter register space
CtrValPtr [out] — Buffer to store the counter value
Return code
Std_ReturnType — E_NOT_OK - Counter retrieval failed
E_OK - Counter successfully retrieved
Functional Description
Retrieves the value of an Ethernet statistics counter.
Particularities and Limitations
Ethernet controller is initialized Function retrieves the value of an Ethernet statistics counter by addressing the counter with the help of an offset into the Ethernet statistics counter register space.
Call context
> ANY > This function is synchronous > This function is reentrant

Table 5-15 Eth_30_Core_GetCounterState

5.3.1.12 Eth_30_Core_ProvideTxBuffer

Prototype	
`BufReq_ReturnType **Eth_30_Core_ProvideTxBuffer** (uint8 CtrlIdx, uint8 *BufIdxPtr, Eth_DataType **BufPtr, uint16 *LenBytePtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
BufIdxPtr [out]	Identifier of the buffer provided on successful buffer provision
BufPtr [out]	Pointer to the buffer provided on successful buffer provision
LenBytePtr [in, out]	Buffer used to determine the requested and the provide length of the buffer: [in] Length of the data the caller wants to transmit (Payload length)
	[out] Actual length of the buffer provided
Return code	
BufReq_ReturnType	BUFREQ_E_NOT_OK - Service was called
	BUFREQ_E_OVFL - No buffer with the requested length available by configuration
	BUFREQ_E_BUSY - Any buffer able to hold the requested length is already in use
	BUFREQ_OK - Buffer successfully provided
Functional Description	
Provides a buffer that can be used to transmit an Ethernet frame.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE Function provides a buffer that can be used to transmit a Ethernet frame. The buffer is locked and therefore protected against reuse until the transmission of the frame is confirmed after transmission was triggered (Eth_Transmit() and consecutive Eth_TxConfirmation()) or buffer is intentionally released by calling Eth_Transmit() with LenByte=0.	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-16 Eth_30_Core_ProvideTxBuffer

5.3.1.13 Eth_30_Core_Transmit

Prototype
`Std_ReturnType **Eth_30_Core_Transmit** (uint8 CtrlIdx, uint8 BufIdx, Eth_FrameType FrameType, boolean TxConfirmation, uint16 LenByte, const uint8 *PhysAddrPtr)`

Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
BufIdx [in]	Identifier of the buffer provided by Eth_ProvideTxBuffer()
FrameType [in]	Ethernet type, according to type field of IEEE802.3
TxConfirmation [in]	Request for a transmission confirmation:
	FALSE - No transmission confirmation desired
	TRUE - Transmission confirmation desired
LenByte [in]	Length of the data to be transmitted (Payload length)
PhysAddrPtr [in]	MAC address the frame shall be sent to
Return code	
Std_ReturnType	E_NOT_OK - Triggering of frame transmission wasn't possible
	E_OK - Frame transmission triggered
Functional Description	
Trigger the transmission of an Ethernet frame created from the passed buffer.	
Particularities and Limitations	
> Ethernet controller is operational in mode ACTIVE > Buffer was acquired by Eth_ProvideTxBuffer() Function takes the buffer previously provided by Eth_ProvideTxBuffer(), enhances it with the Ethernet header and triggers the transmission of the Ethernet frame.	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-17 Eth_30_Core_Transmit

5.3.1.14 Eth_30_Core_Receive

Prototype	
`void Eth_30_Core_Receive (uint8 CtrlIdx, Eth_RxStatusType *RxStatusPtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
RxStatusPtr [out]	Indicates the result of the reception trigger:
	ETH_RECEIVED – Ethernet frame was received, and no more frames are waiting to be received
	ETH_NOT_RECEIVED – No Ethernet frame was received because none was waiting to be received
	ETH_RECEIVED_MORE_DATA_AVAILABLE – Ethernet frame was received and at least one more frame is waiting to be received
	ETH_RECEIVED_FRAMES_LOST – This status is currently not in use

Return code	
void	none
Functional Description	
Triggers the reception of an Ethernet frame.	
Particularities and Limitations	
Calling this function requires the precondition that the Ethernet controller is operational in mode ACTIVE.	
Call context	
> ANY > This function is reentrant > This function is synchronous	

Table 5-18 Eth_30_Core_Receive

5.3.1.15 Eth_30_Core_VTransmit

Prototype	
`Std_ReturnType` **`Eth_30_Core_VTransmit`** `(uint8 CtrlIdx, uint8 BufIdx, Eth_FrameType FrameType, boolean TxConfirmation, uint16 LenByte, const uint8 *PhysAddrPtrDst, const uint8 *PhysAddrPtrSrc)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
BufIdx [in]	Identifier of the buffer provided by Eth_ProvideTxBuffer()
FrameType [in]	Ethernet type, according to type field of IEEE802.3
TxConfirmation [in]	Request for a transmission confirmation: FALSE - No transmission confirmation desired TRUE - Transmission confirmation desired
LenByte [in]	Length of the data to be transmitted (Payload length)
PhysAddrPtrDst [in]	Destination MAC address
PhysAddrPtrSrc [in]	Source MAC address
Return code	
Std_ReturnType	E_NOT_OK - Triggering of frame transmission wasn't possible E_OK - Frame transmission triggered
Functional Description	
Trigger the transmission of an Ethernet frame created from the passed buffer with a specific source MAC.	
Particularities and Limitations	
> Ethernet controller is operational in mode ACTIVE > Buffer was acquired by Eth_ProvideTxBuffer() Function takes the buffer previously provided by Eth_ProvideTxBuffer() enhances it with the Ethernet header (using a specific source MAC address instead of the Ethernet controllers one) and triggers the transmission of the Ethernet frame.	

Call context
> ANY > This function is synchronous > This function is reentrant

Table 5-19 Eth_30_Core_VTransmit

5.3.1.16 Eth_30_Core_TxConfirmation

Prototype	
`void Eth_30_Core_TxConfirmation (uint8 CtrlIdx)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
Return code	
void	none
Functional Description	
Triggers the transmission confirmation of a previously Ethernet frame transmitted.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE Function triggers the transmission confirmation of a previously Ethernet frame transmitted and unlocks the buffer associated to the Ethernet frame, so it can be used for frame transmission again.	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-20 Eth_30_Core_TxConfirmation

5.3.1.17 Eth_30_Core_GetVersionInfo

Prototype	
`void Eth_30_Core_GetVersionInfo (Std_VersionInfoType *VersionInfoPtr)`	
Parameter	
VersionInfoPtr [out]	Buffer to store the version information
Return code	
void	none
Functional Description	
Retrieves the version information of the component.	
Particularities and Limitations	
-	

Function retrieves the Vendor ID, Module ID and software version of the component.
Call context
> ANY > This function is synchronous > This function is reentrant

Table 5-21 Eth_30_Core_GetVersionInfo

5.3.1.18 Eth_30_Core_GetRxStats

Prototype	
`Std_ReturnType` **`Eth_30_Core_GetRxStats`** `(uint8 CtrlIdx, Eth_RxStatsType *RxStats)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
RxStats [out]	List of values to read statistic values for transmission
Return code	
Std_ReturnType	E_OK - success
	E_NOT_OK - Rx-statistics could not be obtained
Functional Description	
Returns list of Reception Statistics.	
Particularities and Limitations	
Function returns the list of Reception Statistics out of IETF RFC1213. Configuration Variant(s): ETH_30_CORE_ENABLE_GET_ETHER_STATS_API	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-22 Eth_30_Core_GetRxStats

5.3.1.19 Eth_30_Core_GetTxStats

Prototype	
`Std_ReturnType` **`Eth_30_Core_GetTxStats`** `(uint8 CtrlIdx, Eth_TxStatsType *TxStats)`	
Parameter	
CtrlIdx [in]	Identifier of the Ethernet controller
TxStats [out]	List of values to read statistic values for transmission
Return code	
Std_ReturnType	E_OK - success

	E_NOT_OK - Tx-statistics could not be obtained
Functional Description	
Returns list of Transmission Statistics.	
Particularities and Limitations	
Function returns the list of Transmission Statistics out of IETF RFC1213. Configuration Variant(s): ETH_30_CORE_ENABLE_GET_ETHER_STATS_API	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-23 Eth_30_Core_GetTxStats

5.3.2 AUTOSAR Extension API

5.3.2.1 Eth_30_Core_GetAndResetMeasurementData

Prototype	
`Std_ReturnType` **`Eth_30_Core_ GetAndResetMeasurementData`** `(uint8 CtrlIdx, Eth_MeasurementIdxType MeasIdx, Boolean ResetNeeded, uint32 *MeasurementData)`	
Parameter	
CtrlIdx [in]	Identifier of the Eth controller
MeasIdx[in]	Identifier of the type of measurement data to be retrieved or reset
	ETH_MEAS_ALL - Only usable if ResetNeeded is TRUE, will reset all measured data
	ETH_MEAS_VENDOR_SPECIFIC_DROP_INSUFF_TX_BUFFER - Reset and/or retrieve amount of denied Tx buffer provision requests due to unavailable buffer resources
	ETH_MEAS_VENDOR_SPECIFIC_WARN_FULL_RX_BUFFER - Reset and/or retrieve amount of warning events which indicate that all Rx buffers are blocked by the software
	ETH_MEAS_VENDOR_SPECIFIC_DROP_INSUFF_RX_BUFFER - Reset and/or retrieve a mount of Rx frames dropped due to unavailable buffer resources
ResetNeeded [in]	Controls if the data determined by MeasIdx should be reset
	FALSE - Reset of the data not requested
	TRUE - Reset of the data requested
MeasurementData [out]	Pointer to store the retrieved measurement data
Return code	
Std_ReturnType	E_OK - Measurement data successfully retrieved or reset
	E_NOT_OK - Measurement data could not be retrieved or reset

Functional Description
This function can be used to retrieve and/or reset the measurement data counter statistics that are supported.
Particularities and Limitations
- Module is initialized
Call context
> TASK|ISR1|ISR2 > This function is synchronous > This function is reentrant

Table 5-24 Eth_30_Core_GetAndResetMeasurementData

5.3.3 Time Synchronization API

5.3.3.1 Eth_30_Core_GetGlobalTime

Prototype	
`Std_ReturnType` **`Eth_30_Core_GetGlobalTime`** `(uint8 CtrlIdx, Eth_TimeStampType *TimestampPtr, Eth_TimestampQualityType *TimestampQualityPtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Eth controller
TimestampPtr [out]	Retrieve time stamp
TimestampQualityPtr [out]	Quality of the time stamp:
	ETH_TIMESTAMP_VALID: Time stamp is valid
	ETH_TIMESTAMP_INVALID: Time stamp is not valid
	ETH_TIMESTAMP_UNCERTAIN: Time stamp is uncertain
Return code	
Std_ReturnType	E_OK - Time stamp successfully retrieved
	E_NOT_OK - Time stamp couldn't be retrieved
Functional Description	
Retrieves the current time of the Eth controller’s timer.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE - Configuration Variant(s): ETH_30_CORE_TIME_SYNC_ENABLED	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-25 Eth_30_Core_GetGlobalTime

5.3.3.2 Eth_30_Core_EnableEgressTimestamp

Prototype	
`Std_ReturnType` **`Eth_30_Core_EnableEgressTimestamp`** `(uint8 CtrlIdx, uint8 BufIdx)`	
Parameter	
CtrlIdx [in]	Identifier of the Eth controller
BufIdx [in]	Identifier of the buffer holding the frame to be transmitted
Return code	
Std_ReturnType	E_OK - Time stamping of frame successfully enabled
	E_NOT_OK - Time stamping of the frame wasn't enabled
Functional Description	
Enables time stamping for the given frame on transmission.	
Particularities and Limitations	
> Ethernet controller is operational in mode ACTIVE > Buffer was previously allocated by Eth_30_Core_ProvideTxBuffer() and no transmission was triggered by Eth_30_Core_Transmit() yet - Configuration Variant(s): ETH_30_CORE_TIME_SYNC_ENABLED	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-26 Eth_30_Core_EnableEgressTimestamp

5.3.3.3 Eth_30_Core_GetEgressTimestamp

Prototype	
`Std_ReturnType` **`Eth_30_Core_GetEgressTimestamp`** `(uint8 CtrlIdx, uint8 BufIdx, Eth_TimeStampType *TimestampPtr, Eth_TimestampQualityType *TimestampQualityPtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Eth controller
BufIdx [in]	Identifier of the buffer used to transmit the frame
TimestampPtr [out]	Time stamp taken on transmission
TimestampQualityPtr [out]	Quality of the time stamp
	ETH_TIMESTAMP_VALID - Time stamp is valid
	ETH_TIMESTAMP_INVALID - Time stamp is not valid
	ETH_TIMESTAMP_UNCERTAIN - Time stamp is uncertain

Return code	
Std_ReturnType	E_OK - Time stamp successfully retrieved
	E_NOT_OK - Time stamp couldn't be retrieved
Functional Description	
Retrieves the time stamp for a frame transmitted.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE - Configuration Variant(s): ETH_30_CORE_TIME_SYNC_ENABLED	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-27 Eth_30_Core_GetEgressTimestamp

5.3.3.4 Eth_30_Core_GetIngressTimestamp

Prototype	
`Std_ReturnType Eth_30_Core_GetIngressTimestamp (uint8 CtrlIdx, Eth_DataType *DataPtr, Eth_TimeStampType *TimestampPtr, Eth_TimestampQualityType *TimestampQualityPtr)`	
Parameter	
CtrlIdx [in]	Identifier of the Eth controller
DataPtr [in]	Memory space the frame is located (as given in <Eth_UL>_RxIndication)
TimestampPtr [out]	Time stamp taken on reception
TimestampQualityPtr [out]	Quality of the time stamp
	ETH_TIMESTAMP_VALID - Time stamp is valid
	ETH_TIMESTAMP_INVALID - Time stamp is not valid
	ETH_TIMESTAMP_UNCERTAIN - Time stamp is uncertain
Return code	
Std_ReturnType	E_OK - Time stamp successfully retrieved
	E_NOT_OK - Time stamp couldn't be retrieved
Functional Description	
Retrieves the time stamp for a frame received.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE - Configuration Variant(s): ETH_30_CORE_TIME_SYNC_ENABLED	
Call context	

> ANY > This function is synchronous > This function is reentrant

Table 5-28 Eth_30_Core_GetIngressTimestamp

5.3.4 FQTSS (Traffic Shaping) API

5.3.4.1 Eth_30_Core_SetBandwidthLimit

Prototype	
`Std_ReturnType` **`Eth_30_Core_SetBandwidthLimit`** `(uint8 CtrlIdx, uint8 QueuePrio, uint32 BandwidthLimit)`	
Parameter	
CtrlIdx [in]	Controller Index
QueuePrio [in]	Queue Priority
BandwidthLimit [in]	Bandwidth limit which shall be assigned for the Tx queue (in [bits/s])
Return code	
Std_ReturnType	E_NOT_OK - New bandwidth limit couldn't be set.
	E_OK - New bandwidth limit set.
Functional Description	
Reconfigures the bandwidth limit set for a transmission queue.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE -	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-29 Eth_30_Core_SetBandwidthLimit

5.3.4.2 Eth_30_Core_GetBandwidthLimit

Prototype	
`Std_ReturnType` **`Eth_30_Core_GetBandwidthLimit`** `(uint8 CtrlIdx, uint8 QueuePrio, uint32 *BandwidthLimitPtr)`	
Parameter	
CtrlIdx [in]	Controller Index
QueuePrio [in]	Queue Priority

BandwidthLimitPtr [out]	Pointer to where to store the currently configured bandwidth limit (in [bit/s])
Return code	
Std_ReturnType	E_NOT_OK - Current bandwidth limit couldn't be retrieved.
	E_OK - Current bandwidth limit retrieved.
Functional Description	
Retrieves the currently configured bandwidth limit of a transmission queue.	
Particularities and Limitations	
Ethernet controller is operational in mode ACTIVE -	
Call context	
> ANY > This function is synchronous > This function is reentrant	

Table 5-30 Eth_30_Core_GetBandwidthLimit

5.4 Services used by Ethernet Driver

In the following table services provided by other components, which are used by the Ethernet Driver are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
Det	`Det_ReportError()`
Dem	`Dem_SetEventStatus()`
EthIf	▶ `EthIf_RxIndication()` ▶ `EthIf_TxConfirmation()`
EthSwt	▶ `EthSwt_EthAdaptBufferLength()` ▶ `EthSwt_EthPrepareTxFrame()` ▶ `EthSwt_EthProcessTxFrame()` ▶ `EthSwt_EthTxFinishedIndication()` ▶ `EthSwt_EthProcessRxFrame()` ▶ `EthSwt_EthRxFinishedIndication()`
NvM (optional)	▶ `NvM_GetErrorStatus()` ▶ `NvM_SetRamBlockStatus()`
Os (optional)	▶ `osReadPeripheral32()` ▶ `osWritePeripheral32()` ▶ `osModifyPeripheral32()`

Table 5-31 Services used by the Ethernet Driver

5.5 Callback Functions

The Ethernet Driver does not provide callback functions.

5.6 Callout Functions

At its configurable interfaces the Ethernet Driver defines callout functions. The declarations of the callout functions are provided by the BSW module, i.e. the Ethernet Driver. It is the integrator's task to provide the corresponding function definitions. The definitions of the callouts can be adjusted to the system's needs. The Ethernet Driver callout function declarations are described in the following tables:

5.6.1 <User>_RxIndicationCallout

Prototype	
`void <User>_RxIndicationCallout (uint8 CtrlIdx, uint16 FrameType, boolean IsBroadcast, const uint8 *PhysAddrPtr, Eth_DataType *DataPtr)`	
Parameter	
`CtrlIdx [in]`	Ethernet controller index
`FrameType [in]`	EtherType the Ethernet frame is related to
`IsBroadcast [in]`	Broadcast indication: FALSE - frame isn't a broadcast frame TRUE - frame is a broadcast frame
`PhysAddrPtr [in]`	Source MAC address of the Ethernet frame
`DataPtr [out]`	Location of the Ethernet frame payload (no VLAN tag considered)
Return code	
`Void`	none
Functional Description	
This optional function to be defined by the user is called instead of the EthIf_RxIndication at the same location, if it is configured and defined.	
Particularities and Limitations	
> User defined	
Call context	
> TASK > ISR1 > ISR2	

Table 5-32 <User>_RxIndicationCallout

5.6.2 <User>_TxConfirmationCallout

Prototype	
`void <User>_TxConfirmationCallout (uint8 CtrlIdx, uint8 BufIdx)`	
Parameter	
`CtrlIdx [in]`	Ethernet controller index

`BufIdx [in]`	Identifier of the buffer holding the frame to be transmitted
Return code	
`Void`	none
Functional Description	
This optional function to be defined by the user is called instead of the EthIf_TxConfirmation at the same location, if it is configured and defined.	
Particularities and Limitations	
> User defined	
Call context	
> TASK > ISR1 > ISR2	

Table 5-33 <User>_TxConfirmationCallout

5.6.3 Appl_Eth_30_<Driver>_MemoryBarrier

Prototype	
`void` **`Appl_Eth_30_<Driver>_MemoryBarrier`** `(void)`	
Parameter	
void	
Return code	
void	void
Functional Description	
The function shall provide a platform and compiler specific memory barrier so that following memory and peripheral register accesses are synchronized.	
Particularities and Limitations	
none	
Call context	
> Task > This function is Synchronous > This function is Reentrant > Availability: Always	

Table 5-34 Appl_Eth_30_<Driver>_MemoryBarrier

6 Configuration

The Ethernet Driver is configured with the help of the following Vector tools:

> DaVinci Configurator PRO

6.1 Configuration Variants

The Ethernet Driver supports the configuration variants

> VARIANT-PRE-COMPILE

The configuration classes of the Ethernet Driver parameters depend on the supported configuration variants. For their definitions please see the Eth_30_<Driver>_bswmd.arxml file.

6.2 Driver specific configuration

For configuration aspects of the specific driver please see [6].

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
DaVinci Configurator PRO	Generation tool for MICROSAR components

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
FCS	Frame Check Sequence
FIFO	First In First Out
FQTSS	Forwarding and Queuing of Time Sensitive Streams
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MAC	Media Access Control
MCU	Microcontroller Unit
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
MII	Media Independent Interface
NvM	Non volatile (RAM) Manager
QoS	Quality of Service
RAM	Random Access Memory
RTE	Runtime Environment
RTOS	Real-Time Operating System
RX/Rx	Reception
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification
TX/Tx	Transmission

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com